STM32控制器及开发工具

目录

CONTENTS

01 ARM概述

02 STM32 Cortex-M4控制器

03 开发工具

知识、能力、素质目标

知识目标

- 阐述ARM相关概念、寄存器和指令集
- 阐述STM32 Cortex-M4控制器特性
- 设计STM32F407核心板
- 运用STM32程序设计软件及仿真软件

能力目标

- 运用C语言在STM32开发中的编程能力
- 设计STM32F407最小系统硬件电路的能力
- 综合STM32CubeMX、Keil MDK、Proteus进行STM32设计、调试的能力

素质目标

- 能够主动收集、整理文献资料，能够自主学习。
- 保持对新技术、新知识的关注和学习热情，紧跟STM32生态系统的发展。

01

ARM概述

1.1 引言

1. ARM有3层含义：处理器名称，全称为高级精简指令集计算机机器（Advanced RISC Machine），是低功耗、低成本的精简指令集计算机微处理器；公司名称，全称为Advanced RISC Machines Limited；技术名称，以高效、低功耗为特点的高级RISC技术。

2. RISC与复杂指令集计算机（Complex Instruction Set Computer，CISC）相比，在指令集简化与执行效率、硬件设计与功耗、寄存器与内存访问、流水线技术、编译优化与程序执行效率以及可靠性与成本等方面都展现出明显的优势。

1.1 引言

3. 1991年，ARM 公司成立于英国剑桥，目前，采用ARM公司技术知识产权核的微处理器，即通常所说的ARM微处理器，已遍及工业控制、消费类电子产品、通信系统、网络系统、无线系统等各类产品市场。目前，众多集成电路制造商推出了ARM结构芯片，我国的中兴集成电路、中科芯集成电路、华为海思、平头哥半导体、极海半导体、上海灵动微电子和中微半导体等，以及国外的德州仪器、意法半导体（STMicroelectronics，ST）、飞利浦、三星等都推出了设计的基于ARM核的处理器。

1.2 基于ARM体系结构划分

ARM体系结构是一个复杂而强大的处理器架构，主要包括微处理器所支持的指令集和基于该体系结构下微处理器的编程模型。ARM指令集是处理器结构中最重要的部分，用于操作CPU。

1. ARM系列分类

ARM公司自2004年推出ARMv7内核架构后，开始使用以Cortex命名的架构系列，包括Cortex-A、Cortex-R和Cortex-M共3个主要系列。

Cortex-A

面向性能要求高的系统应用，如智能手机、平板计算机、汽车娱乐系统等。它支持高计算要求、运行丰富的操作系统以及提供优质的交互媒体和图形体验。

Cortex-R

面向实时应用的高性能内核，适用于需要快速响应和可靠性的实时嵌入式系统，如汽车制动系统、大容量存储控制器等。

Cortex-M

面向各类嵌入式应用的微控制器内核，具有低功耗和高性能的特点。它分为多个子系列，主要应用于工控嵌入式系统、物联网设备、智能测量、人机接口设备等。

图2-1 32位ARM Cortex内核产品

2. 指令集

在ARM架构中，ARM指令和Thumb指令是两种重要的指令集，它们在指令长度、执行效率、代码密度等方面存在显著差异，以满足不同应用场景的需求。

1）ARM指令

◆ARM指令以32位为单位进行编码，每条指令包含多个字段，用于指定操作码、寄存器和立即数等信息。

◆ARM指令集广泛应用于需要高性能计算的嵌入式系统和移动设备中。

- Thumb指令是ARM指令集的一个变种，采用16位的指令长度，相较于ARM指令更加紧凑。Thumb指令集通过减少指令的存储空间和内存带宽消耗，实现了更高的代码密度，特别适用于资源受限的嵌入式系统和移动设备。
- Thumb-2指令集结合了ARM指令和Thumb指令的优点，既可以使用紧凑的16位指令，也可以使用功能更强大的32位指令，这使得Thumb-2指令集既能满足资源受限环境下的性能需求，又能支持复杂计算和数据处理任务。

1.3 ARM处理器中的寄存器

ARM处理器共有**37个32位寄存器**，其中**31个**为通用寄存器，**6个**为状态寄存器，用于存储和传输数据、指令以及处理器的状态信息。

ARM处理器共有7种不同的处理器模式：

- 用户模式（User Mode，usr）
- 系统模式（System Mode，sys）
- 快速中断模式（Fast Interrupt Request Mode，fiq）
- 管理模式（Supervisor Mode，svc）
- 中止模式（Abort Mode，abt）
- 外部中断模式（Interrupt Request Mode，irq）
- 未定义模式（Undefined Mode，und）

1. 通用寄存器

通用寄存器包括R0~R15，它们可以分为以下3类。

1）未分组寄存器（R0~R7）

在所有运行模式（处理器模式）下，未分组寄存器都指向同一个物理寄存器。

2）分组寄存器（R8~R14）

对于分组寄存器R8~R12，每个寄存器对应两个不同的物理寄存器。一组用于除FIQ模式外的所有处理器模式，而另一组则专门用于FIQ模式。

不同模式下寄存器的使用，要使用**寄存器名后缀**加以区分。例如，当使用FIQ模式下的寄存器时，寄存器R8和寄存器R9分别记为R8_fiq、R9_fiq；当使用用户模式下的寄存器时，寄存器 R8 和 R9分别记为R8_usr、R9_usr等。

对于分组寄存器 R13 和R14 来说，每个寄存器对应6个不同的物理寄存器。其中的一个是用户模式和系统模式共用的，而另外5 个分别用于5 种异常模式。访问时需要指定它们的模式，名字形式如下R13_<mode>、R14_<mode>，其中<mode>可以是usr、svc、abt、und、irp及fiq中模式之一。

3）程序计数器（Program Counter，PC）（R15）

寄存器R15用作程序计数器存储当前执行的指令的地址。在ARM状态下，位[1：0]为0，位[31：2]用于保存PC；在Thumb状态下，位[0]为0，位[31：1]用于保存PC。

2. 状态寄存器

状态寄存器包括当前程序状态寄存器（Current Program Status Register，CPSR）和备份的程序状态寄存器（Saved Program Status Register，SPSR）。

CPSR可以在任何处理器模式下被访问，用于控制指令的执行状态、中断的允许和禁止等。

SPSR在每一种处理器模式下都有一个专用的物理寄存器。当特定的异常中断发生时，SPSR用于保存CPSR的当前值。当异常处理程序返回时，再将其内容恢复到CPSR。

表2-1 寄存器与ARM处理器7种不同模式的对应关系

寄存器类别	寄存器名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器	R0	R0						
	R1	R1						
	R2	R2						
	R3	R3						
	R4	R4						
	R5	R5						
	R6	R6						
	R7	R7						
	R8	R8						R8_fiq
	R9	R9						R9_fiq
	R10	R10						R10_fiq
	R11	R11						R11_fiq
	R12	R12						R12_fiq
	R13	R13	R13_svc	R13_abt		R13_und	R13_irq	R13_fiq
	R14	R14	R14_svc	R14_abt		R14_und	R14_irq	R14_fiq
	R15	R15						
状态寄存器	CPSR	CPSR						
	SPSR	无	SPSR_svc	SPSR_abt		SPSR_und	SPSR_irq	SPSR_fiq

1.4 ARM处理器特点

ARM处理器具有优良性能，作为一种先进的RISC处理器，ARM处理器有如下特点。

1. 低功耗

ARM处理器采用了多种节能设计，如动态电源管理、指令集优化等，使其在运行过程中能够保持较低的功耗。这一特点使其非常适合于移动设备和物联网设备等对功耗有严格要求的场景。ARM处理器在提供高性能的同时，能够保持较低的功耗，这使得设备在长时间运行或电池供电时具有更长的续航时间。

1.4 ARM处理器特点

2. 高性能

ARM处理器采用RISC架构，通过精简指令集来降低指令的复杂性和执行时间，从而提高了处理器的性能。

3. 小尺寸

ARM处理器的物理尺寸较小，这使得它非常适合于尺寸要求严格的设备，如智能手机、平板电脑等便携式设备。

4. 广泛的生态系统

ARM处理器得到了众多软件开发商、操作系统厂商和硬件制造商的支持，形成了庞大的生态系统。

1.4 ARM处理器特点

5. 可扩展性

ARM架构支持多种处理器核心和配置，可以根据不同的应用需求进行定制和优化。这使得ARM处理器能够适应从低功耗嵌入式设备到高性能计算设备的广泛需求。

6. 安全性

ARM架构注重安全性设计，提供了硬件级别的安全特性，这些技术可以保护敏感数据和代码的安全，满足对安全性要求较高的应用场景。

02

STM32 Cortex-M4控制器

2.1引言

STM32 Cortex系列32位微控制器是由意法半导体公司基于ARMv7架构的内核设计和生产的微型控制单元（Microcontroller Unit，MCU）。

图2-2 STM32芯片

STM32 Cortex-M4控制器主要为基于ARM Cortex-M4内核的高性能微控制器STM32F4系列，其广泛应用于工业自动化、消费电子、汽车电子、医疗设备、航空航天以及物联网设备等多个领域。

2.2 STM32 Cortex-M4控制器特性

1. 概述

STM32 Cortex-M4是带有数字信号处理和浮点运算单元指令的STM32F4系列高性能微控制器。

STM32 Cortex- M4控制器内部集成了高速存储器（高达1 MB的闪存、高达128 KB的SRAM）、用于静态存储器的灵活外部存储器控制器（FSMC）以及Quad SPI闪存接口等。

STM32 Cortex-M4控制器提供了多种外设接口，包括多达3个快速12位ADC（5Msps）、两个比较器、两个运算放大器、两个DAC通道、一个内部电压基准缓冲器、一个低功耗RTC以及多个通用定时器和低功耗定时器等。

2.2 STM32 Cortex-M4控制器特性

表2-2 STC2产品型号含义

序号	符号	含义
1	STM32	芯片系列，基于ARM的32位微控制器
2	F	产品类型，通用型
3	407	芯片子系列
4	V	引脚数量，R-64引脚，O-90引脚，V-100引脚，Z-144引脚，I-176引脚
5	E	Flash内存容量，E-512K，G-1024K
6	T	封装形式，T-LQFP，H-UFBGA，Y-WLCSP
7	6	温度范围，6代表40℃~85℃，7代表-40℃~105℃

2.2 STM32 Cortex-M4控制器特性

2. STM32 Cortex-M4控制器总线

Cortex-M4内核中高级高性能总线（Advanced High-performance Bus，AHB）和高级外设总线（Advanced Peripheral Bus，APB）是两种重要的总线架构，它们分别承担着不同的功能和性能需求。

AHB是一种高性能、高吞吐量的系统总线，分为AHB1、AHB2、AHB3种总线，主要用于连接处理器核心、高速存储器和高带宽外设等高性能模块。

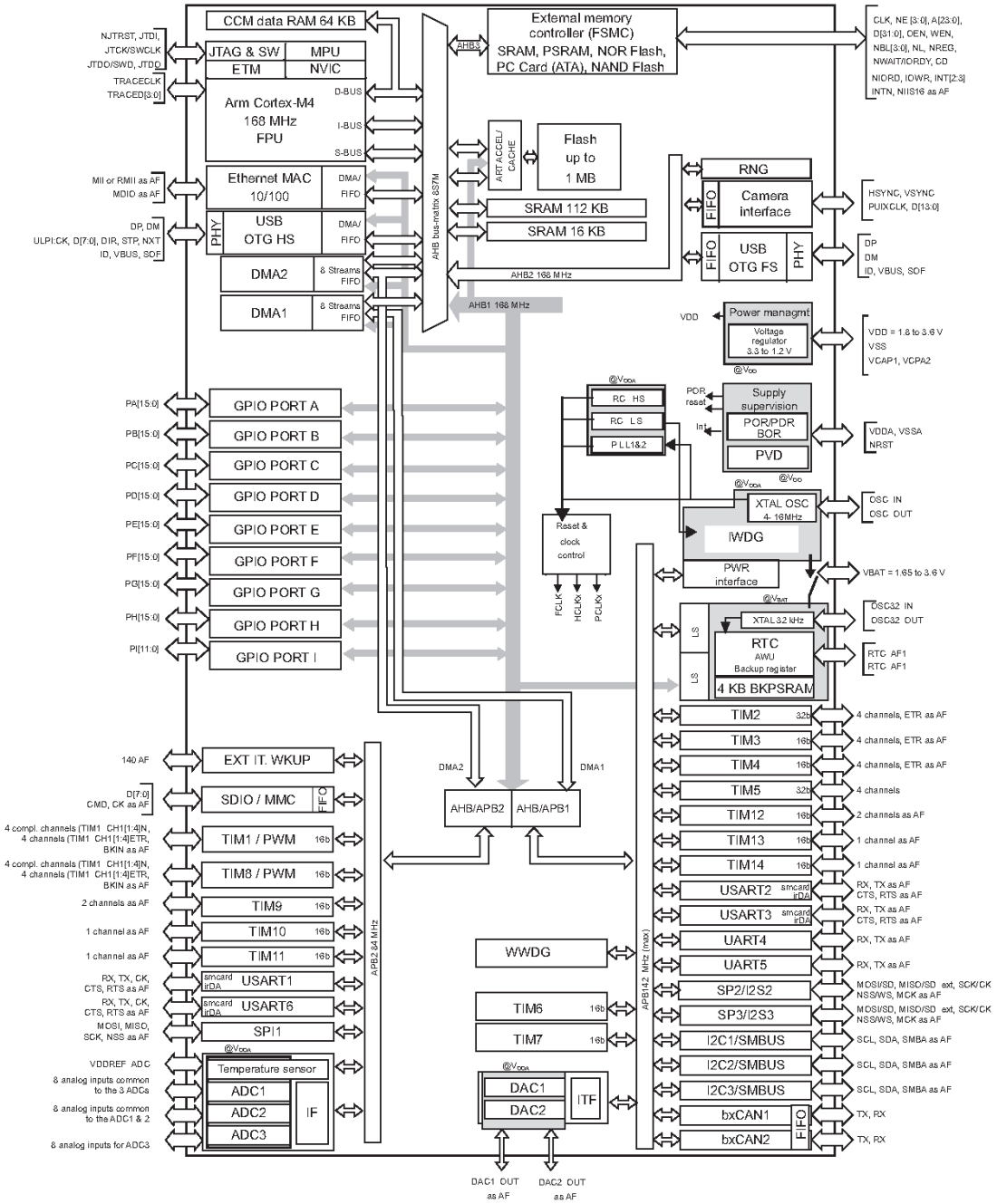


图2-3 Cortex-M4内核的STM32F4系列微控制器功能框图

2.2 STM32 Cortex-M4控制器特性

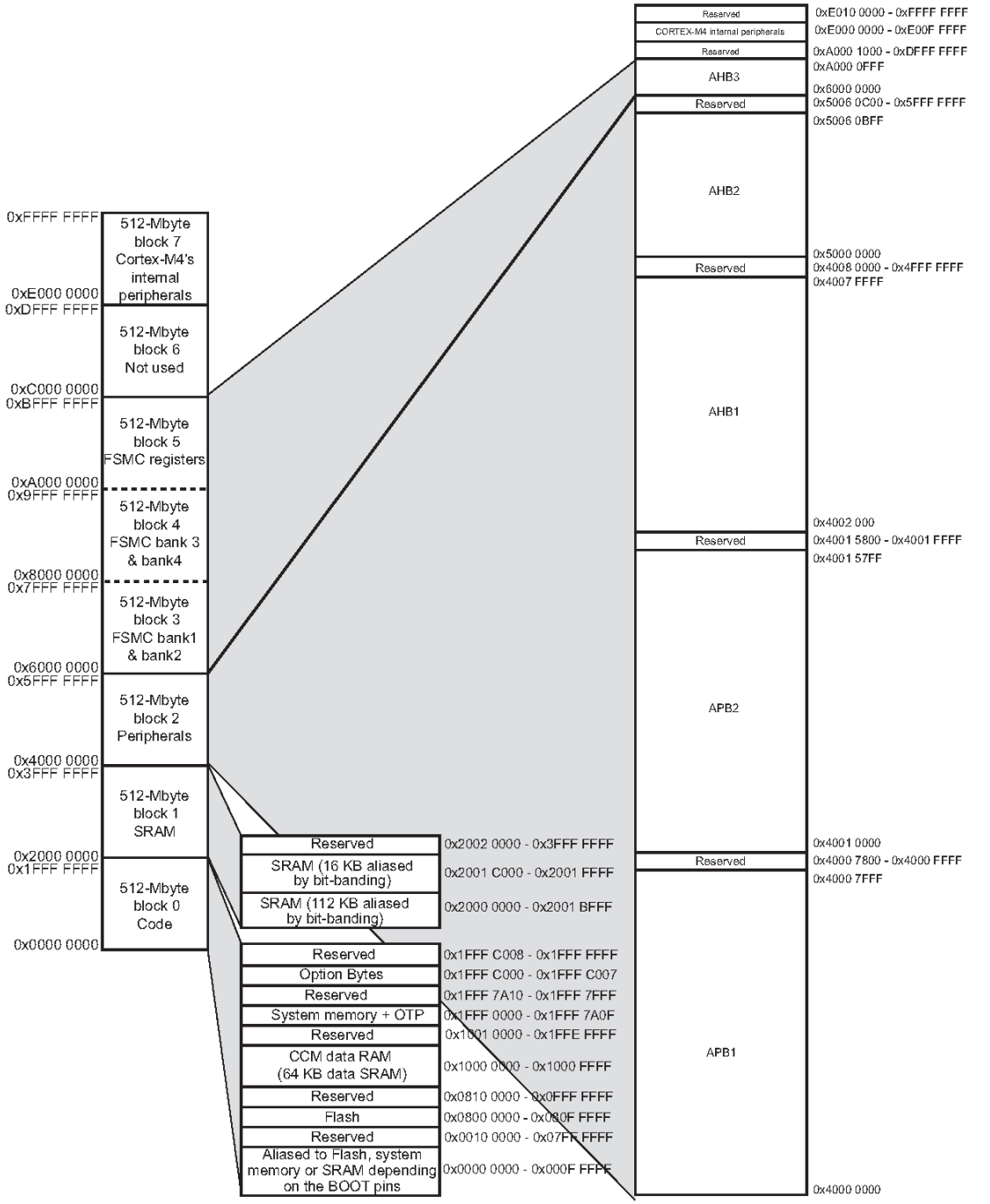
2. STM32 Cortex-M4控制器总线

在STM32 Cortex-M4系列中，APB通常分为APB1和APB2两种，以适应不同速度需求的外设连接。

APB1的速度较慢，适合连接对速度要求不高的外设。APB1架构中，APB桥是唯一的主模块，负责与其他从模块（外设）的通信。APB1连接的外设包括串口（USART/UART）、I2C、SPI等低速设备。

APB2通常连接速度相对较高的外设，如高级定时器（如TIM）、串行接口（如USART1、USART6）、模数转换器（Analog-to-Digital Converter，ADC）、I/O端口以及某些高速通信接口（如通用串行总线（Universal Serial Bus，USB））。

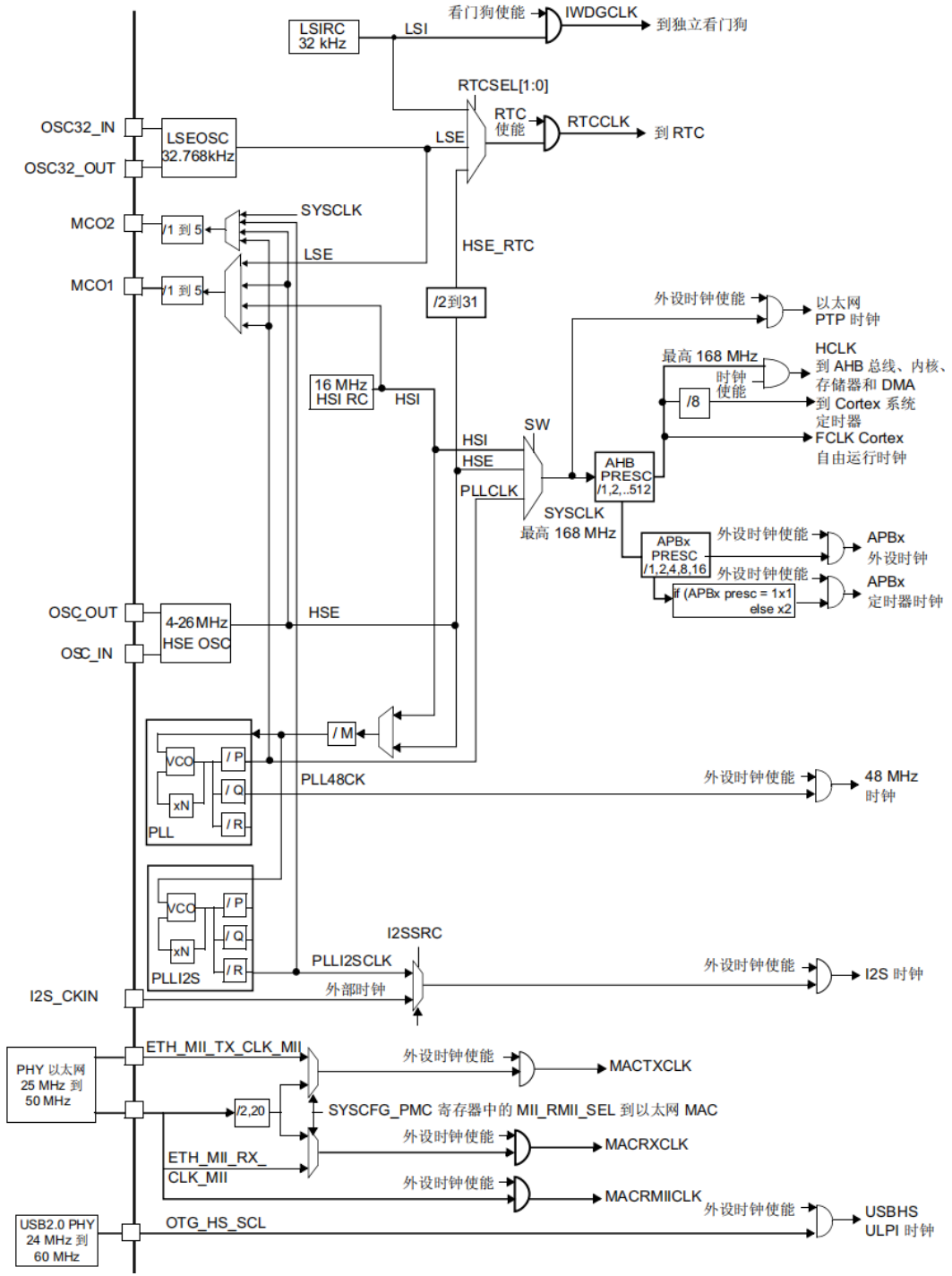
AHB和APB在STM32F4系列微控制器中扮演着至关重要的角色。它们通过高效、灵活的数据传输和控制信号传递机制，确保了处理器与内部存储器、高速外设以及低速外设之间的无缝连接和协同工作。



2.2 STM32 Cortex-M4控制器特性

3. 时钟系统

STM32 Cortex-M4的时钟系统是一个复杂而强大的系统，为微控制器提供各种不同的时钟频率，以满足处理器核、外设以及总线系统的需求。STM32 Cortex-M4系列微控制器拥有多个时钟源和分频器，能够灵活地为系统提供所需的时钟频率。时钟系统在嵌入式系统中至关重要，直接影响微控制器及外设的稳定性和性能。合理的时钟系统设计能够降低功耗、提升系统稳定性和运行效率。


LSIRC
32 kHz
LSI
看门狗使能
IWDGCLK
到独立看门狗
RTCSEL[1:0]
OSC32_IN
OSC32_OUT
LSEOSC
32.768kHz
LSE
RTC
使能
RTCCLK
到 RTC
MCO2
/1 到 5
SYSCLK
LSE
HSE_RTC
MCO1
/1 到 5
/2到31
外设时钟使能
以太网
PTP 时钟
最高 168 MHz
HCLK
到 AHB 总线、内核、
存储器和 DMA
时钟
使能
/8
到 Cortex 系统
定时器
FCLK Cortex
自由运行时钟
16 MHz
HSI RC
HSI
SW
HSI
HSE
PLLCLK
AHB
PRESC
/1,2,..512
SYSCLK
最高 168 MHz
APBx
PRESC
/1,2,4,8,16
外设时钟使能
APBx
外设时钟
外设时钟使能
APBx
定时器时钟
if (APBx presc = 1x1
else x2
OSC_OUT
OSC_IN
4-26 MHz
HSE OSC
HSE
/ M
VCO
/ P
/ Q
xN
/ R
PLL
PLL48CK
外设时钟使能
48 MHz
时钟
VCO
/ P
/ Q
xN
/ R
PLLI2S
I2SSRC
PLLI2SCLK
外部时钟
外设时钟使能
I2S 时钟
I2S_CKIN
PHY 以太网
25 MHz 到
50 MHz
ETH_MII_TX_CLK_MII
外设时钟使能
MACTXCLK
/2,20
SYSCFG_PMC 寄存器中的 MII_RMII_SEL 到以太网 MAC
外设时钟使能
MACRXCLK
ETH_MII_RX_
CLK_MII
外设时钟使能
MACRMIICLK
USB2.0 PHY
24 MHz 到
60 MHz
OTG_HS_SCL
外设时钟使能
USBHS
ULPI 时钟

2.2 STM32 Cortex-M4控制器特性

其中PLL实际分为两个时钟源，分别为主PLL和专用PLL。从时钟频率来分可以分为高速时钟源和低速时钟源，在这5个中HSI、HSE及PLL是高速时钟，LSI和LSE是低速时钟。从来源可分为外部时钟源和内部时钟源，外部时钟源就是通过外接晶振的方式获取时钟源，其中HSE和LSE是外部时钟源，其他的是内部时钟源。

2.2 STM32 Cortex-M4控制器特性

在STM32 Cortex-M4中，有5个最重要的时钟源：

- LSI是低速内部时钟，RC振荡器，频率为32kHz左右。供独立看门狗和自动唤醒单元使用。
- LSE是低速外部时钟，接频率为32.768kHz的石英晶体。这个主要是RTC的时钟源。
- HSE是高速外部时钟，可接石英/陶瓷谐振器，或者接外部时钟源，频率范围为4MHz-26MHz。
- SI是高速内部时钟，RC振荡器，频率为16MHz。可以直接作为系统时钟或者用作PLL输入。
- PLL为锁相环倍频输出。

2.2 STM32 Cortex-M4控制器特性

4. 启动流程及模式

（1）系统复位：STM32复位后，首先会从地址0x00000000处取出堆栈指针MSP的初始值（栈顶地址）。

（2）读取复位向量：接着从地址0x00000004处取出程序计数器指针PC的初始值（复位向量），该值指向复位处理函数Reset_Handler的入口地址。

2.2 STM32 Cortex-M4控制器特性

（3）执行复位处理函数：跳转到Reset_Handler函数，该函数通常负责调用SystemInit函数进行系统初始化，并最终跳转到C库的__main函数。

（4）初始化用户堆栈：__main函数会进行一系列初始化工作，包括初始化用户堆栈，为调用main函数做准备。

（5）执行用户主函数：最后，__main函数会调用用户编写的main函数，从而开始执行用户程序。

STM32 Cortex-M4的启动模式主要由BOOT引脚（BOOT0和BOOT1）的配置决定。这些引脚在系统复位后的系统时钟（System Clock，SYSCLK）的第4个上升沿被锁存，从而确定了STM32的启动模式，STM32 Cortex-M4启动模式设置条件如下。

1） 从主闪存存储器启动

启动条件：BOOT1为任意值（通常为0），BOOT0为0。

启动地址：0x08000000。这是STM32内置的Flash存储器的起始地址，也是最常见的启动模式。在开发过程中，通常使用JTAG或SWD

2.2 STM32 Cortex-M4控制器特性

2） 从系统存储器启动

启动条件：BOOT1为0，BOOT0为1。

启动地址：0x1FFF0000。系统存储器是芯片内部一块特定的区域，STM32在出厂时，由制造商在这个区域内部预置了一段BootLoader，即ISP程序。这种启动模式通常用于通过串口或其他方式下载程序到Flash中，因为系统存储器中的BootLoader提供了串口下载程序的固件。但需要注意的是，使用这种启动模式后，需要手动将BOOT0设置回0，并重新复位，才能使STM32从Flash中启动。

2.2 STM32 Cortex-M4控制器特性

3） 从内置静态存储器启动

启动条件： BOOT1为1，BOOT0为1。

启动地址：0x20000000。SRAM通常用于程序调试，因为它允许在不擦除Flash的情况下快速加载和测试代码。然而，由于SRAM在断电后会丢失数据，因此它不适用于长期存储程序代码。

表2-3 STM32 Cortex-M4启动模式设置

启动模式选择引脚		启动模式
BOOT1	BOOT0	
X	0	主闪存存储器启动
0	1	系统存储器启动
1	1	内置静态存储器启动

2.2 STM32 Cortex-M4控制器特性

5. STM32 Cortex- M4控制器特点

强大的处理器核心

STM32 Cortex-M4控制器搭载了高性能的ARM Cortex-M4 32位RISC内核，工作频率高达80MHz。该内核具有浮点处理单元，支持单精度浮点运算，支持所有ARM单精度数据处理指令和数据类型，同时还实现了全套DSP指令和增强应用程序安全性的内存保护单元（Memory Protection Unit，MPU）。

丰富的外设接口

STM32 Cortex-M4控制器提供了多种外设接口，包括通用输入输出、ADC、数模转换器、定时器、通用串行接口等，以满足不同应用需求。此外，它还支持多种通信协议和接口，如以太网、USB、CAN等，方便与其他设备或网络连接。

低功耗设计

STM32 Cortex-M4控制器具有多种低功耗模式，包括睡眠模式、停机模式等，适用于对功耗有严格要求的应用场景。

内存管理

STM32 Cortex-M4控制器具有不同容量的闪存和RAM，支持存储程序代码和数据，满足不同复杂度的程序需求。同时，它还提供了多种保护机制。

集成度高

STM32 Cortex-M4控制器将多个功能集成在单一芯片上，减少了外部组件的需求，有助于缩小系统体积和降低成本。

可靠性和实时性

STM32 Cortex-M4控制器适用于需要高可靠性和实时响应的应用，如工业控制和汽车电子系统。其强大的处理能力和丰富的外设接口使得它能够快速响应各种事件并做出相应处理。

2.2 STM32 Cortex-M4控制器特性

6. STM32 M4控制器应用领域

STM32 M4控制器在多个领域都有广泛的应用。

01

工业自动化:在工厂自动化、机器人控制、传感器接口和数据采集等方面表现出色，帮助实现高效的工业自动化系统。

02

消费电子:在智能手机、平板计算机、家庭娱乐系统、数字相机和音频设备等消费电子产品中得到广泛应用。

03

汽车电子:无论是发动机控制单元、车身电子系统，还是车载娱乐系统和驾驶员辅助系统，控制器都能凭借其高性能和可靠性，确保车辆的安全性和功能的高度集成。

6. STM32 M4控制器应用领域

04

医疗设备:在医疗设备中发挥着重要作用，如心电图仪、血压计、血糖仪和医疗图像处理等。其高精度和低功耗特性使得这些设备更加可靠和便携。

05

航空航天:在航空航天领域也扮演着重要角色，用于飞行控制、航空电子仪器、导航系统和航空通信等。其高性能和实时性特点使得航空航天系统更加安全和可靠。

06

物联网（Internet of Things，IoT）设备:适用于各种物联网设备，如智能传感器、智能家居网关和工业物联网设备等。其低功耗和长续航能力使得这些设备在物联网应用中具有显著优势。

2.3 STM32F407核心板

- 两个USB OTG（其中一个支持高速传输）
- 音频：专用音频PLL和两个全双工I^2S
- 通信接口多达15个
- 模拟：两个12位DAC、3个12位ADC
- 定时器17个：频率最高168 MHz的16和32位定时器
- 可以利用支持Compact Flash、SRAM、PSRAM、NOR和NAND存储器的灵活静态存储器控制器扩展存储容量。
- 基于模拟电子技术的真随机数发生器。
- 512KB~1MB Flash和192KB SRAM。

2.3 STM32F407核心板

STM32核心板主要由微控制器芯片、复位电路、时钟电路、电源电路、程序调试和下载接口五部分组成。

1. STM32F407VET6芯片

STMicroelectronics生产的高性能、低功耗的32位ARM Cortex-M4微控制器。STM32F407VET6的核心部件是ARM Cortex-M4内核，采用了Thumb-2指令集，支持16位和32位指令，具有较高的运算能力和代码密度。Cortex-M4内核具有FPU和DSP功能，支持所有ARM单精度数据处理指令和数据类型，满足各种嵌入式应用的需求。

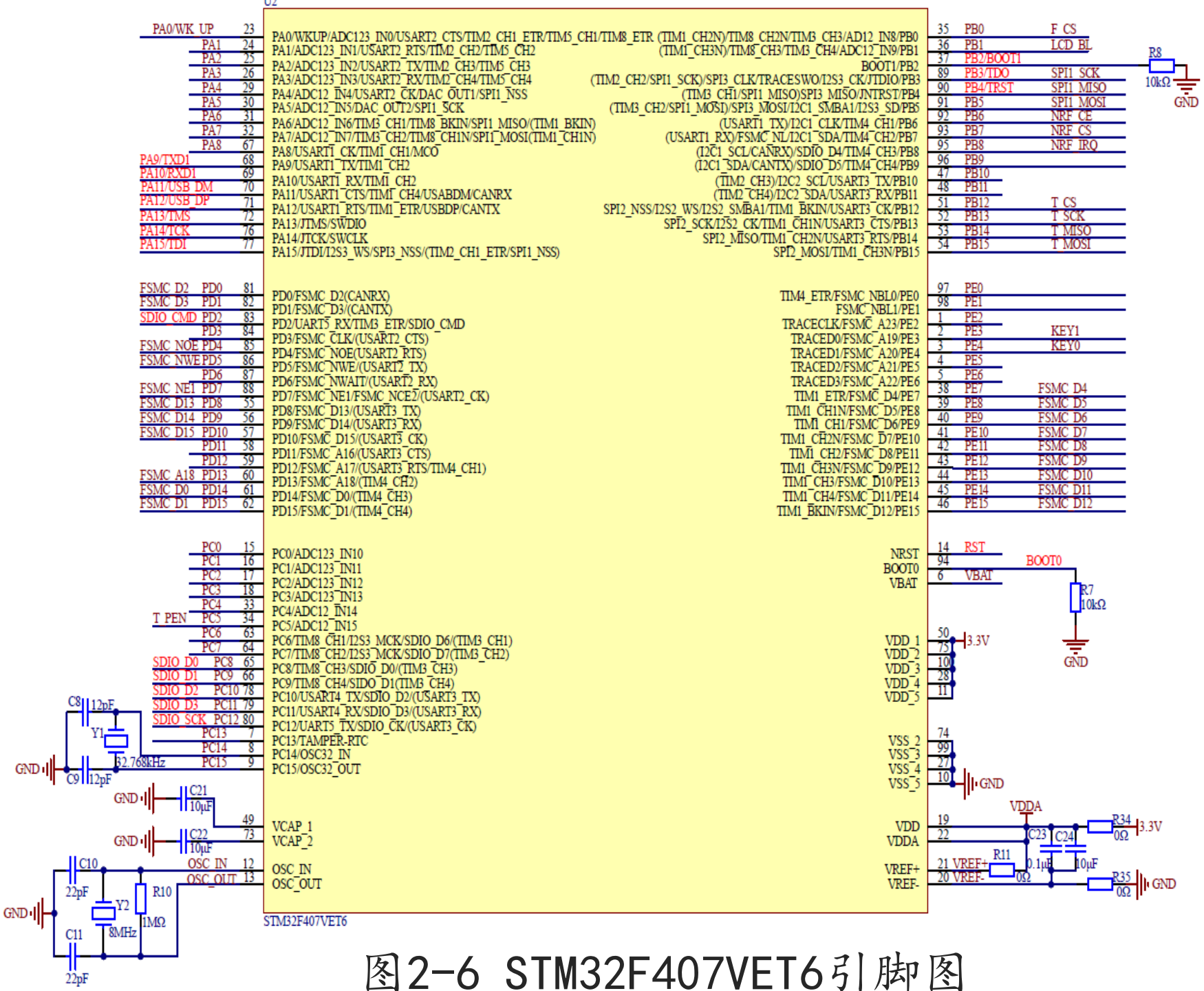


图2-6 STM32F407VET6引脚图

2. 复位电路

STM32F407VET6核心板复位电路如图2-7所示，图中R4和C12构成了上电复位电路。SW1为手动复位按键。除了时钟控制寄存器中的复位标志和备份域中的寄存器外，系统复位会将其他全部寄存器都复位为复位值。

只要发生以下事件之一，就会产生系统复位：

- NRST 引脚低电平。
- 窗口看门狗计数结束。
- 独立看门狗计数结束。
- 软件复位。
- 低功耗管理复位。

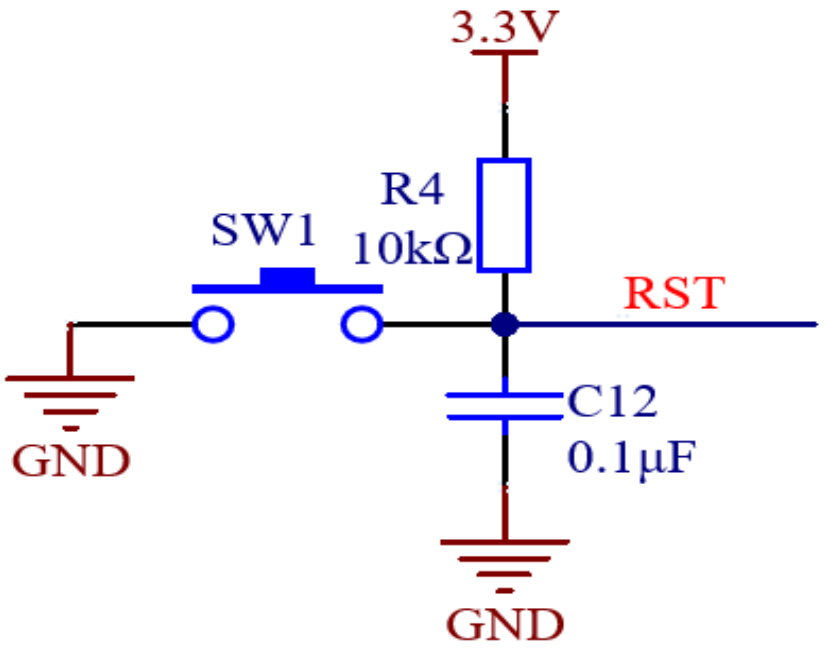


图2-7 复位电路图

3. 时钟电路

在实际电路中，STM32F407系列微控制器外接8MHz HSE和32.768kHz LSE时钟，在未使用时都可单独打开或者关闭，以降低功耗。时钟电路图如图2.8所示。

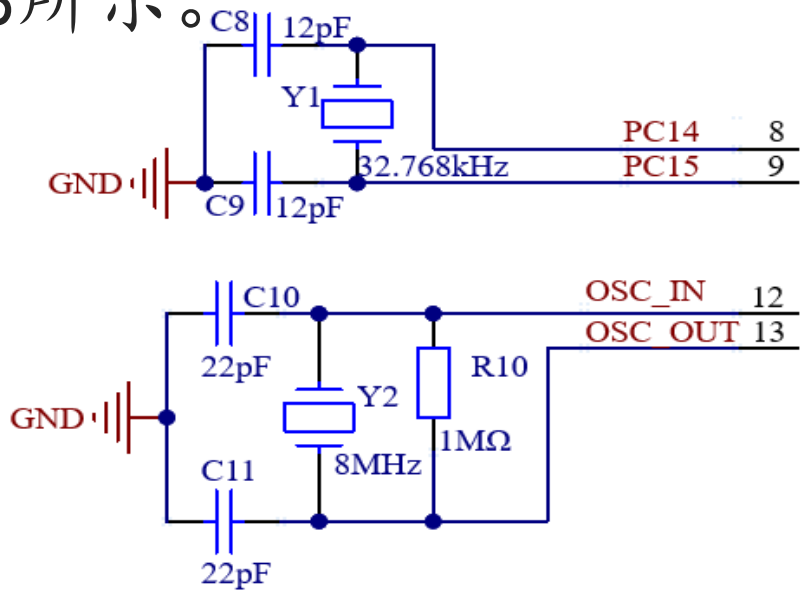


图2-8 时钟电路图

HSE通过PLL倍频，可以生成最高168MHz的系统时钟。LSE用于驱动实时时钟（Real Time Clock，RTC）等功能，RTC在STM32F407中是一个独立的时钟，即使在系统主时钟关闭时也能继续运行。

4. 电源供电电路

电源供电电路是用于核心板供电的单元，应确保各个电压不同的单元都能正常工作。

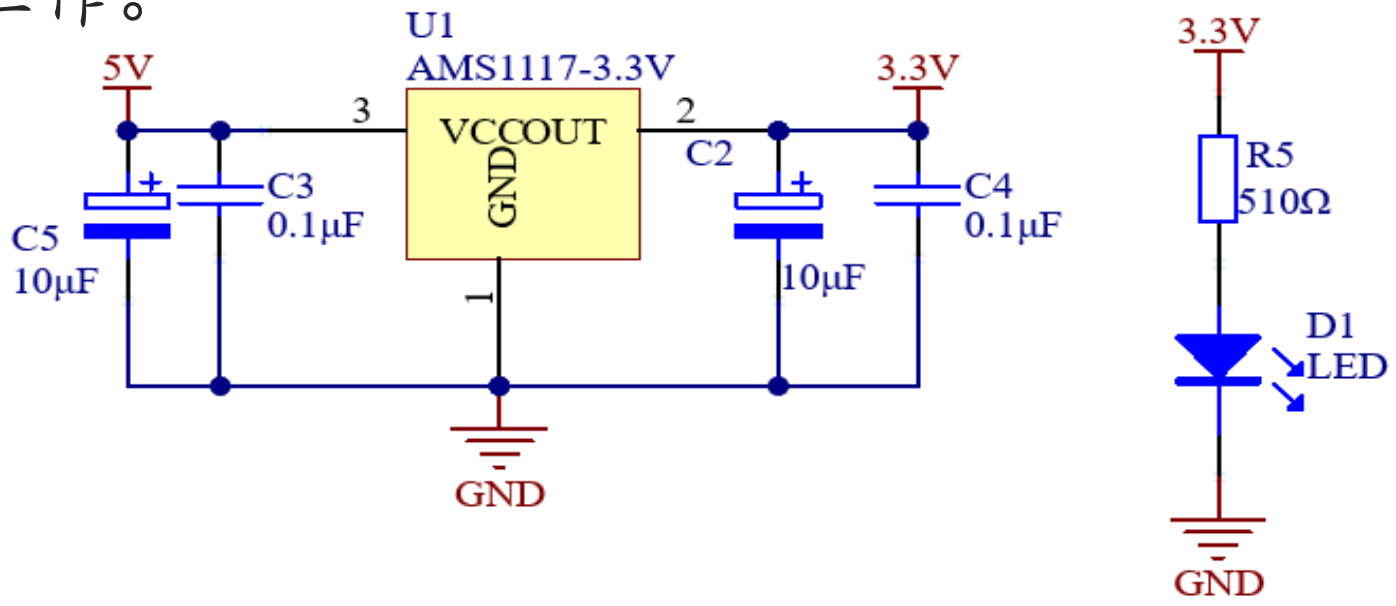


图2-9　核心板的供电电路原理图

AMS1117为电压转换芯片，将5V供电电压转换为芯片需要的3.3V，R5为限流电阻，D1为发光二极管。

5. 程序调试和下载接口

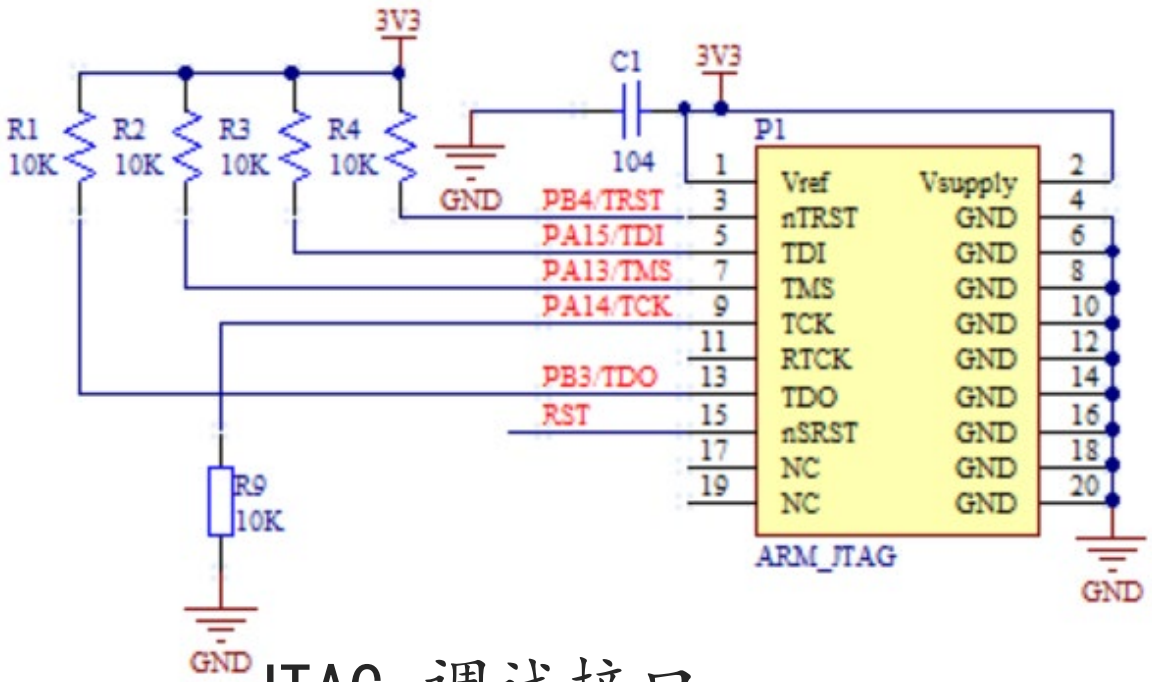
STM32F407的程序下载有多种方法：JTAG、SWD、USB转串口等。

表2-4 JTAG与SWD引脚定义

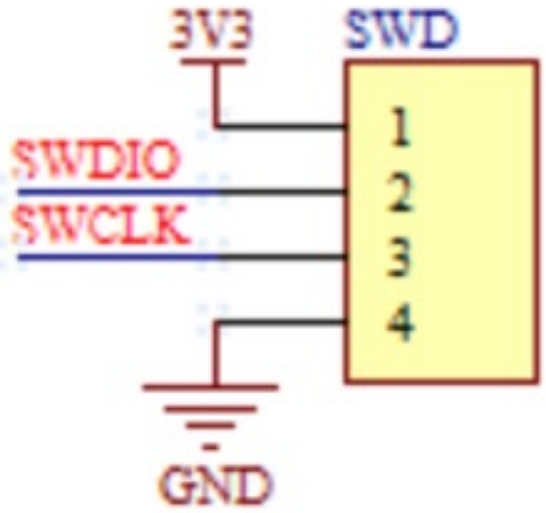
引脚名称	JTAG调试端口		SWD调试端口		引脚分配
	类型	说明	类型	调试分配	
TMS/SWDIO	I	JTAG测试模式设置	I/O	串行线数据输入/输出	PA13
TCK/SWCLK	I	JTAG测试时钟	I	串行线时钟	PA14
TDI	I	JTAG测试数据输入	-	-	PA15
TDO/SWO	O	JTAG测试数据输出	-	TRACESWO（如果使能异步跟踪）	PB3
TRST	I	JTAG测试复位	-	-	PB4

5. 程序调试和下载接口

SWD一般只需要4个引脚或5个引脚，结构简单。还可采用USB转串口进行程序下载，通常采用CH340G的芯片使计算机的USB映射为串口使用。



JTAG 调试接口



SWD串行接口

图2-10 调试下载接口电路

03

开发工具

3.1 STM32CubeMX

1. 概述

ST公司提供了丰富的开发工具和软件支持，如STM32CubeMX、STM32CubeIDE等。STM32CubeMX采用简单易用的图形界面，可以快速配置硬件和软件，并生成适用的STM32平台的C代码项目。

STM32CubeMX可以实现的功能包括：MCU/MPU、开发板、引脚、外设和中间件配置、时钟配置、Keil、IAR和STM32CubeIDE项目生成、功耗估算、软件包管理器。

3.1 STM32CubeMX

2. 软件安装

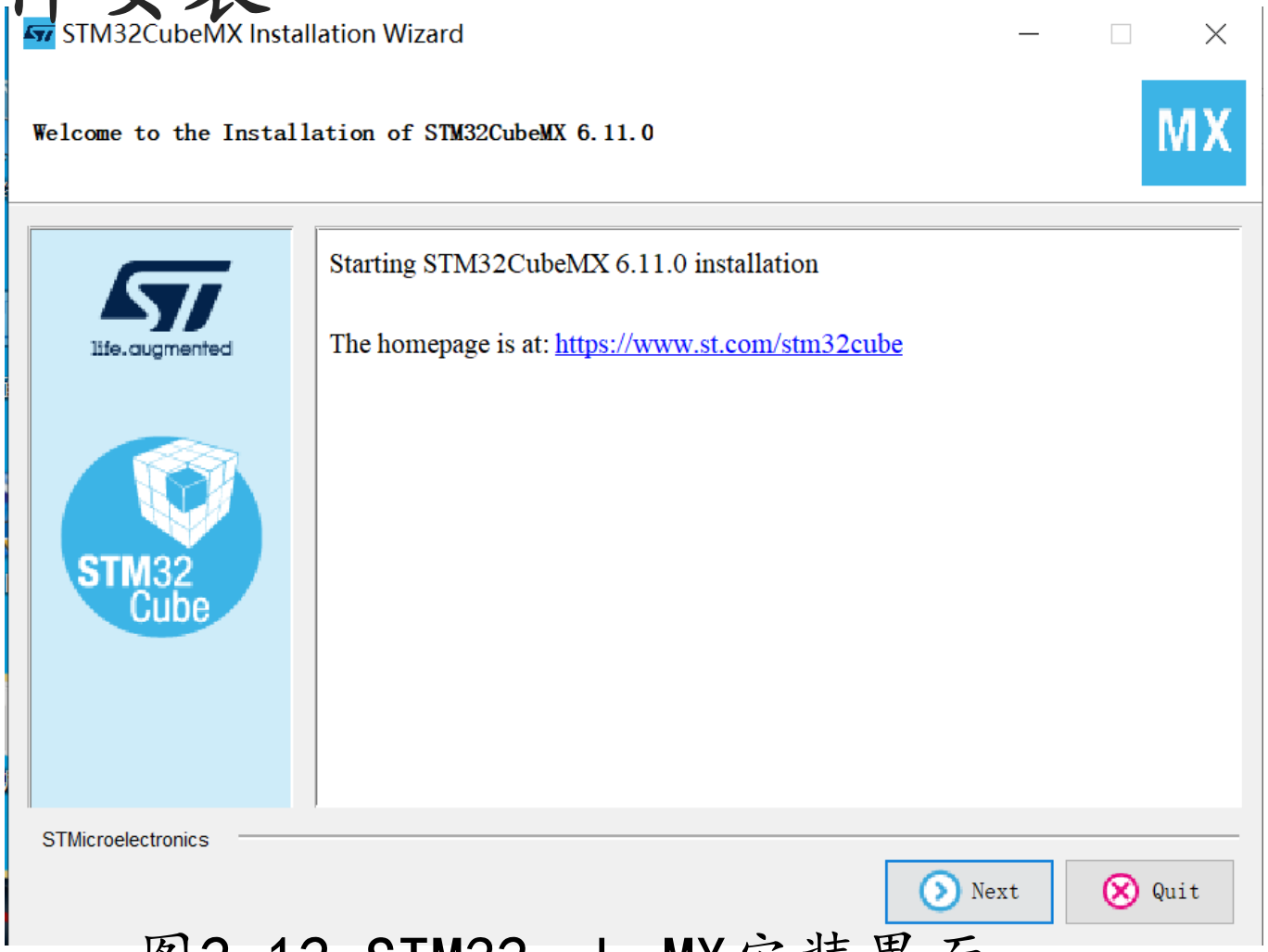
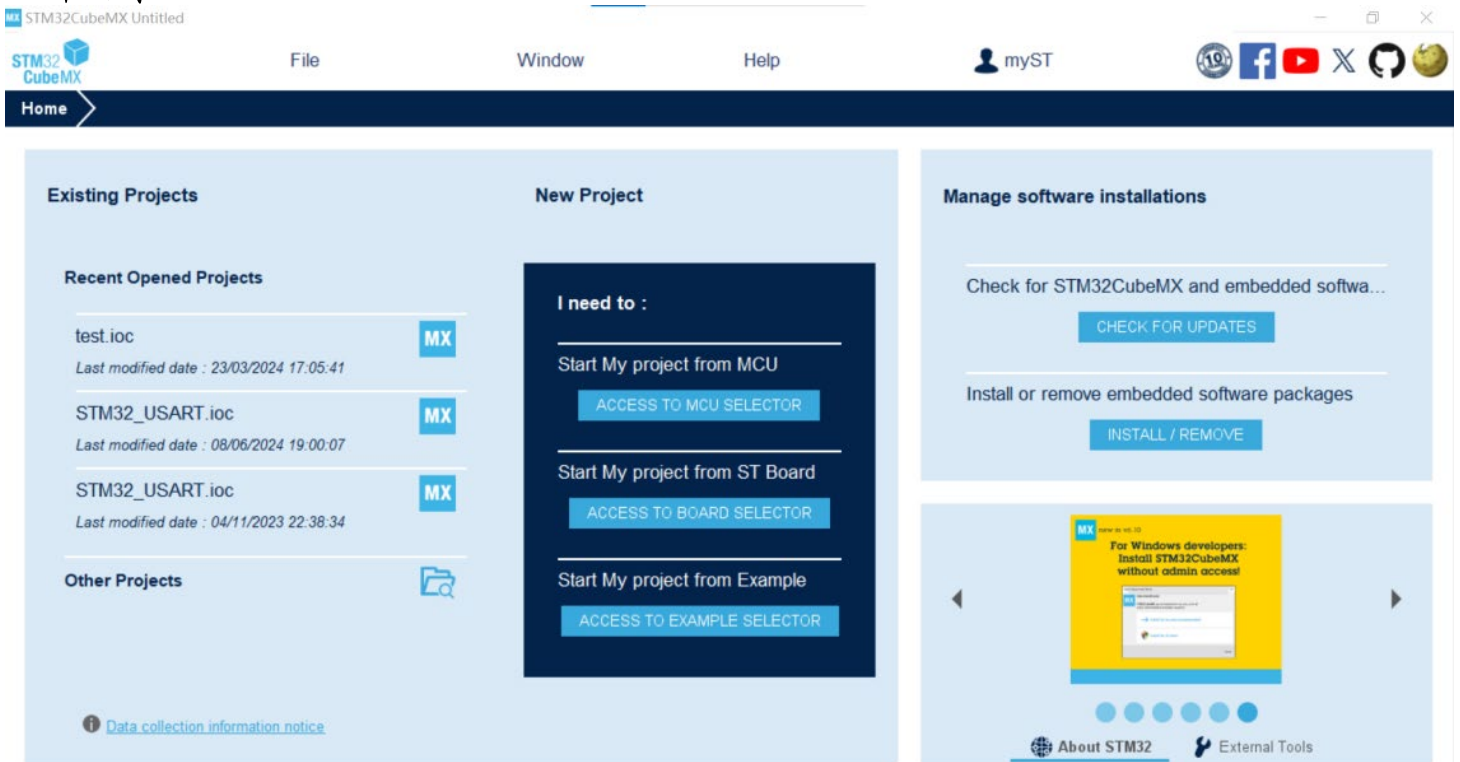


图2-12 STM32cubeMX安装界面

3.1 STM32CubeMX

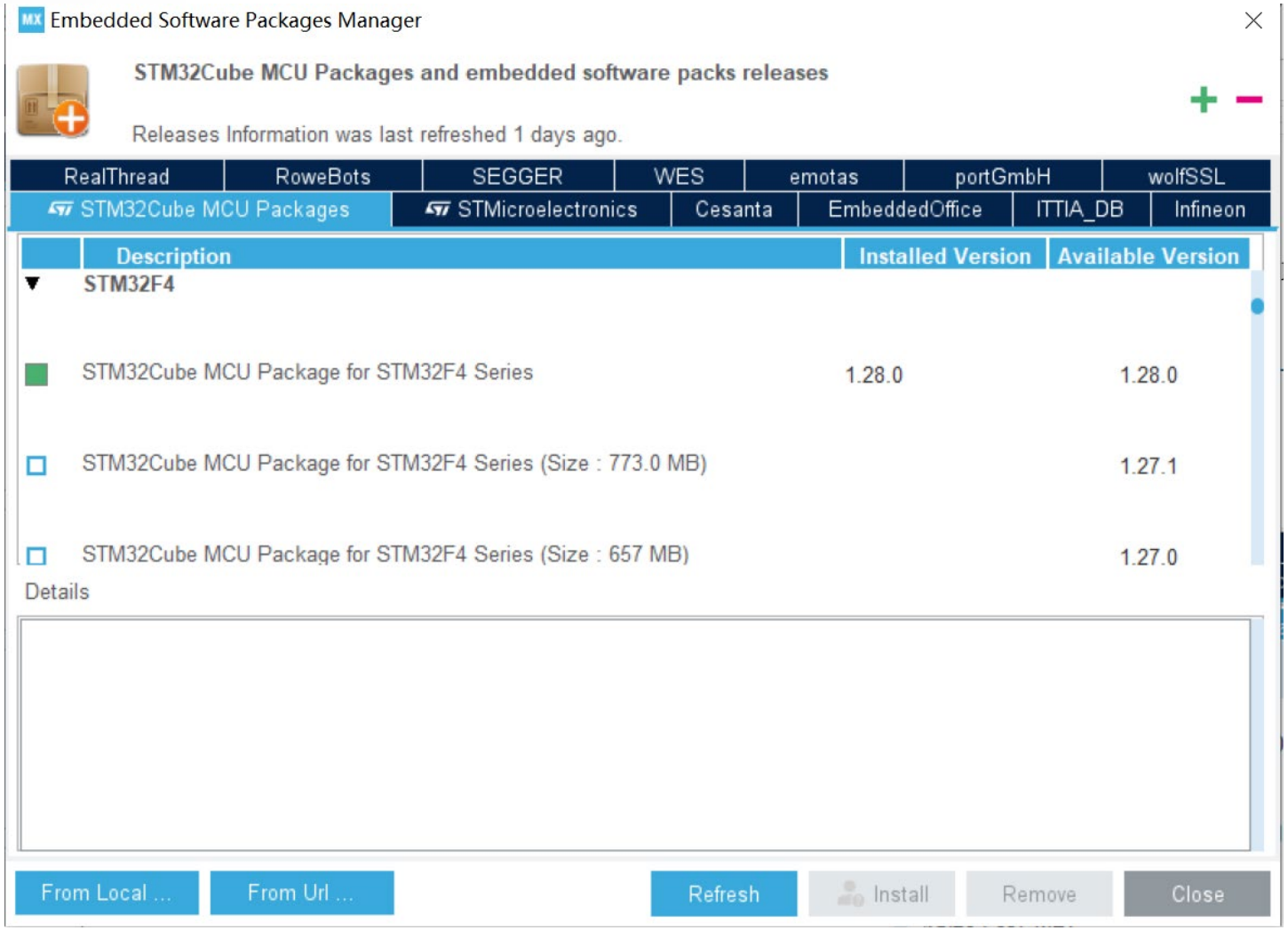
3. HAL库安装

安装完成后，以管理员权限运行STM32CubeMX软件，打开STM32CubeMX软件界面。



3.1 STM32CubeMX

单击菜单栏 Help->Manage embedded software packages，选中安装 STM32Cube MCU Packages for STm32F4 Series，单击 Install，硬件抽象层（Hardware Abstraction Layer，HAL）库安装界面。



3. HAL库安装

HAL是ST公司为STM32微控制器提供的固件库，可以有效提高开发效率和可移植性。HAL库是硬件抽象层库，为开发者提供了一个标准的接口，使其能够以更加独立的方式操作硬件，提高软件的重用性和移植性，简化固件开发，让开发者更加关注应用层逻辑，而无须深入了解底层硬件细节。

3.1 STM32CubeMX

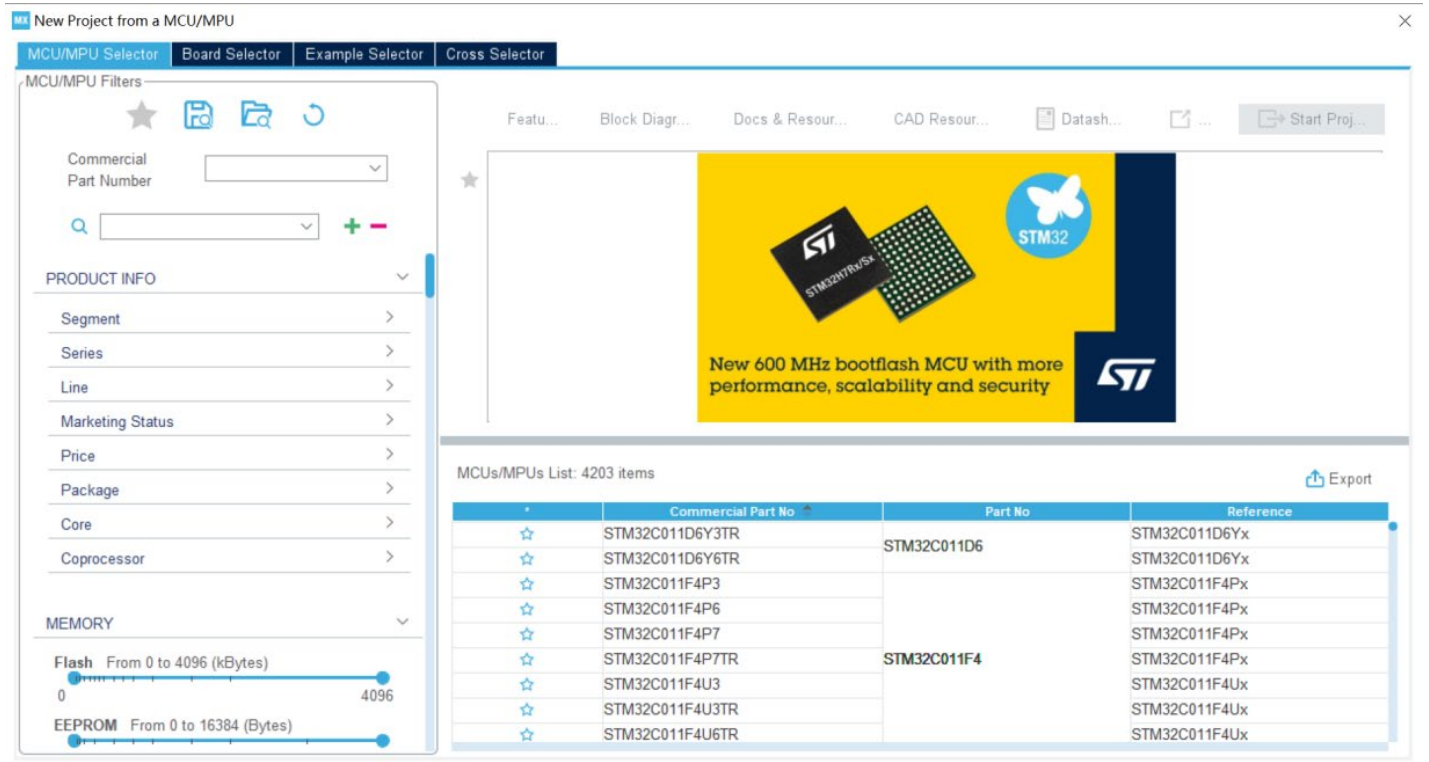
HAL库中以**_HAL开头函数**是HAL库中的底层或内部函数，这些函数通常是被HAL_函数或其他内部函数所调用，以实现特定的硬件操作或功，比如__HAL_RCC_GPIOA_CLK_ENABLE（）函数其作用是使能GPIOA时钟，通常被GPIO_Init（）函数调用。

以**HAL_**开头的函数是HAL库中的高层API，用于提供对STM32微控制器硬件功能的封装和抽象，这些函数通常以HAL_为前缀，后跟具体的功能名称，如HAL_GPIO_Init（）用于初始化GPIO引脚，

HAL_函数的设计使得可以更加便捷地访问和控制硬件，而无需深入了解底层的硬件细节。

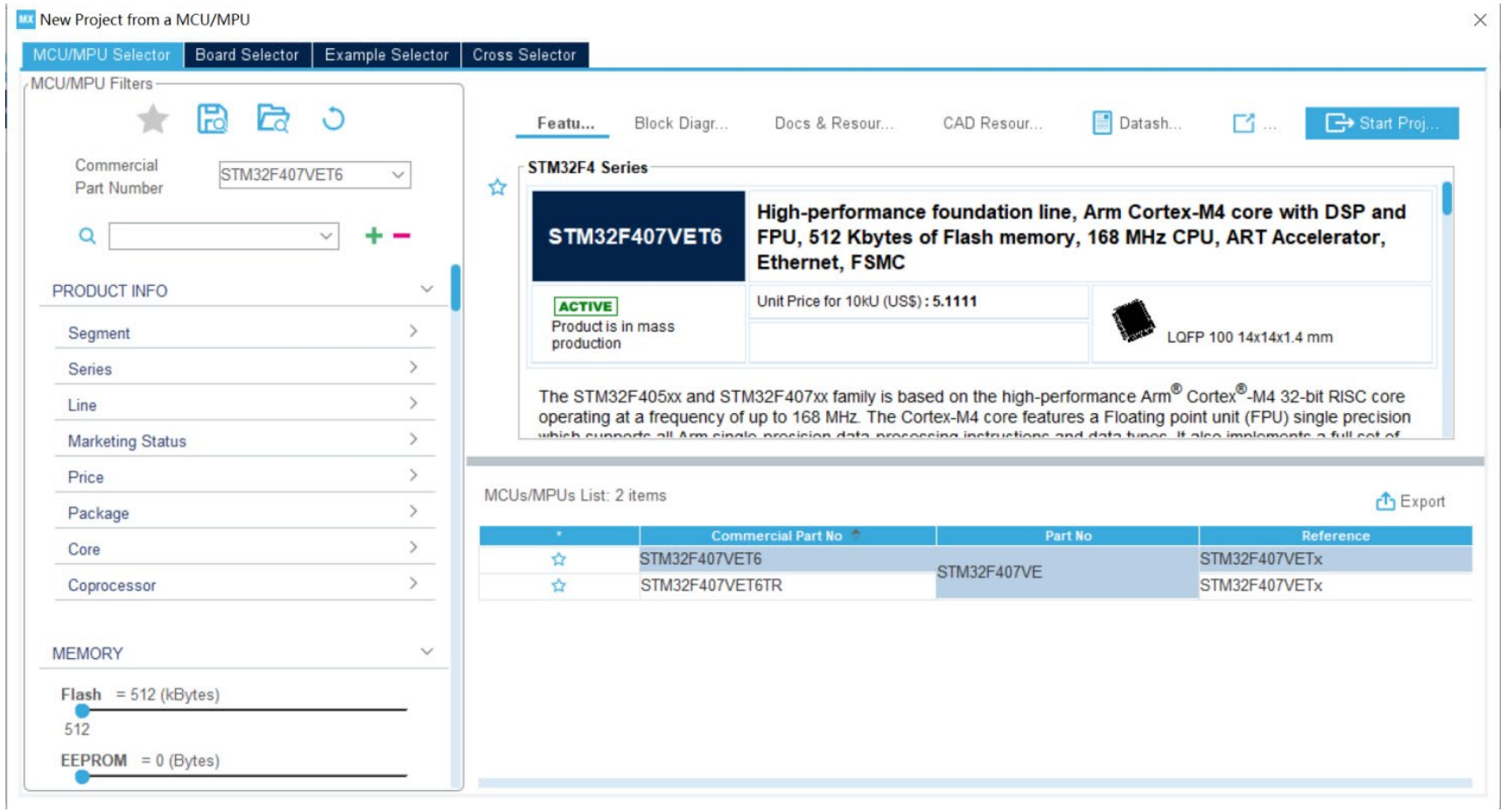
4. 新建STM32CubeMX工程

MCU/MPU Selector（MCU/MPU选择器）、Board Selector（开发板选择器）、Example Selector（例程选择器）、Cross Selector（交叉选择器）



3.1 STM32CubeMX

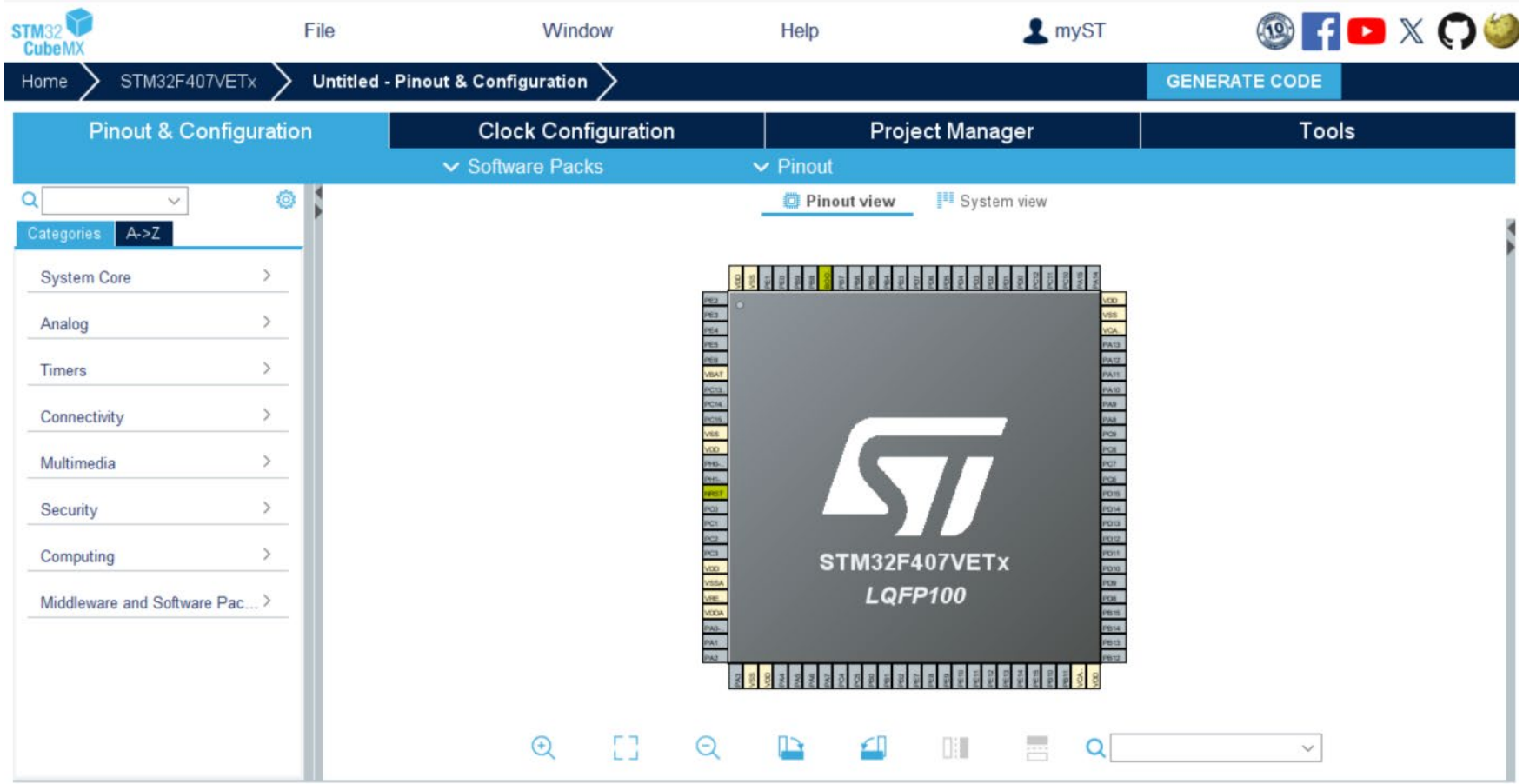
选择MCU/MPU Selector新建工程，在Commercial Part Number栏输入STM32F407VET6。



3.1 STM32CubeMX

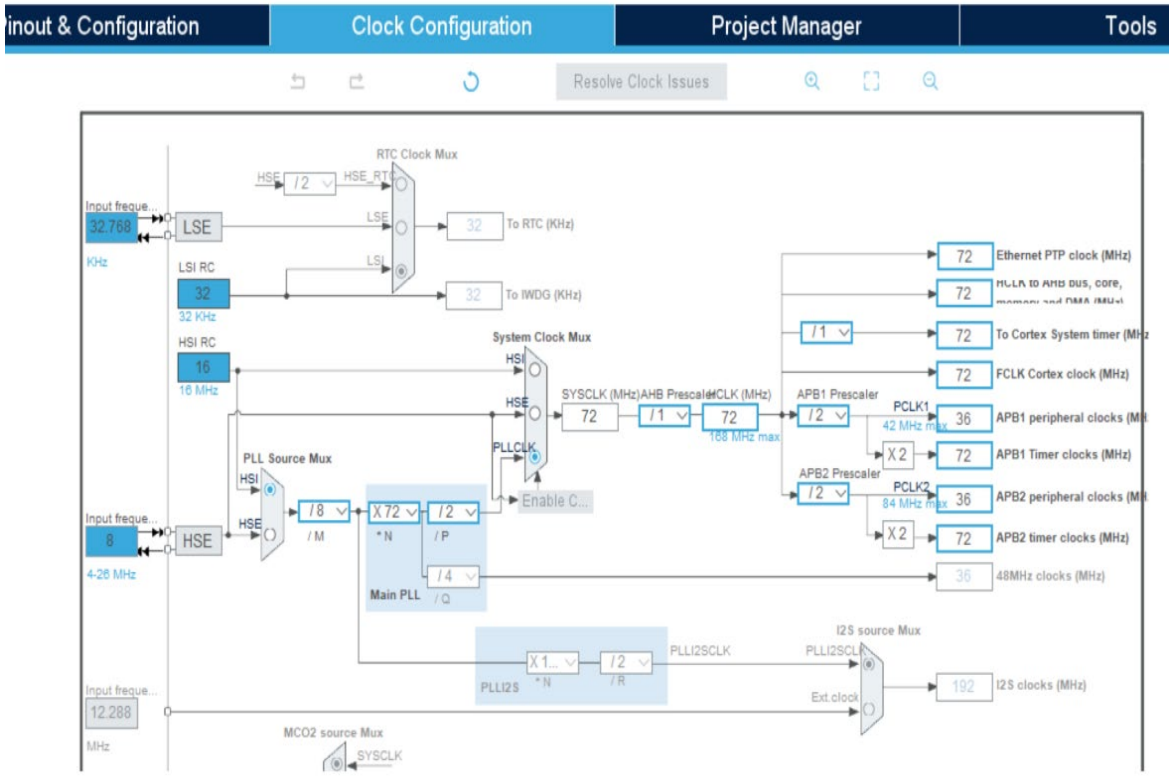
5. 芯片配置

STM32CubeMX有芯片配置图形化界面，可简化整个系统的芯片功能配置。



6. 配置时钟

STM32CubeMX有可视化完整时钟树、时钟配置和自动解算器，可根据具体需求完成解析。单击Clock Configuration选项页。

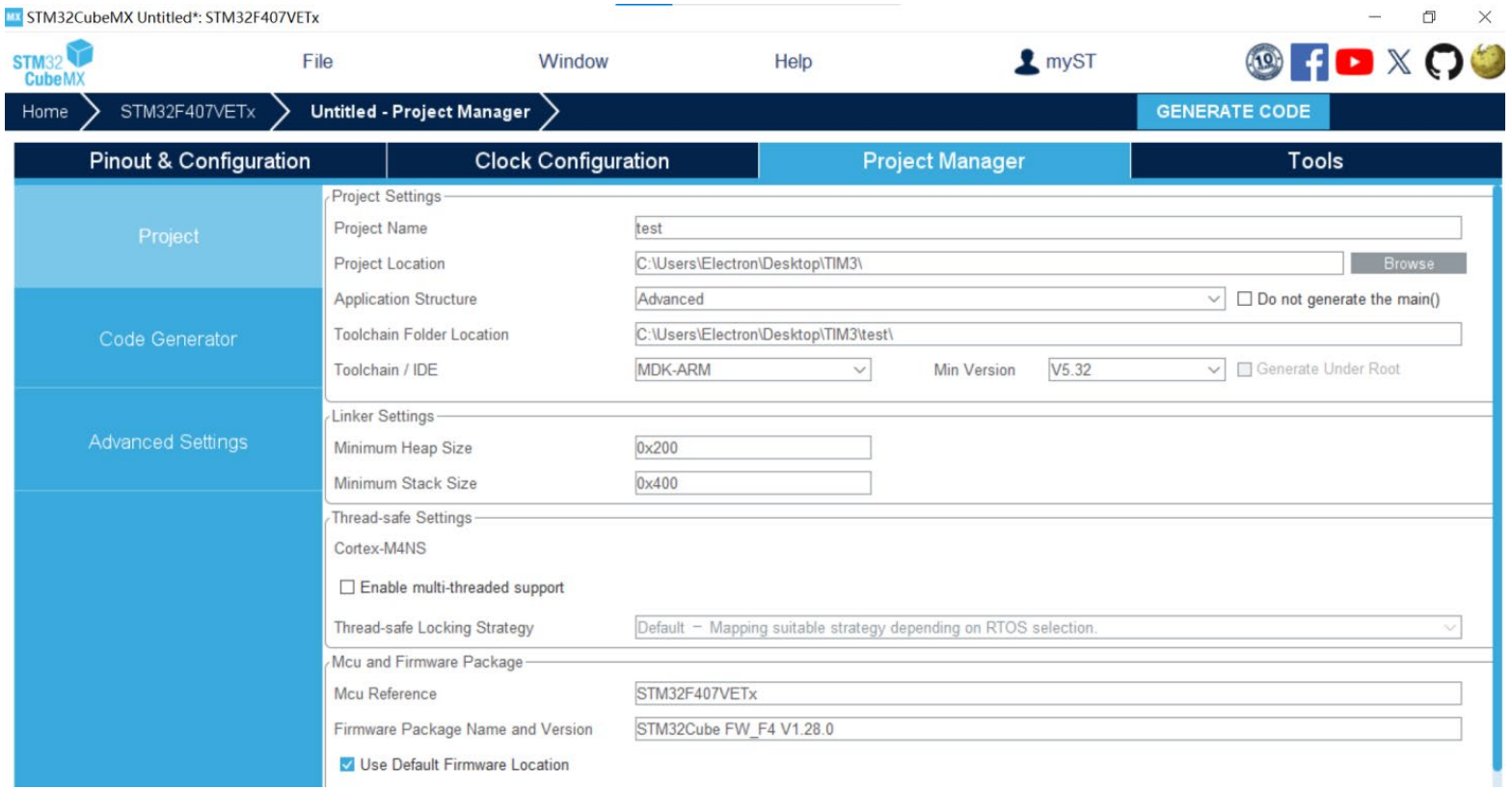


左侧8MHz时钟为外部高速时钟，32.768KHz为外部低速时钟，通过时钟配置图可以方便地配置系统各单元时钟。

7. 工程管理

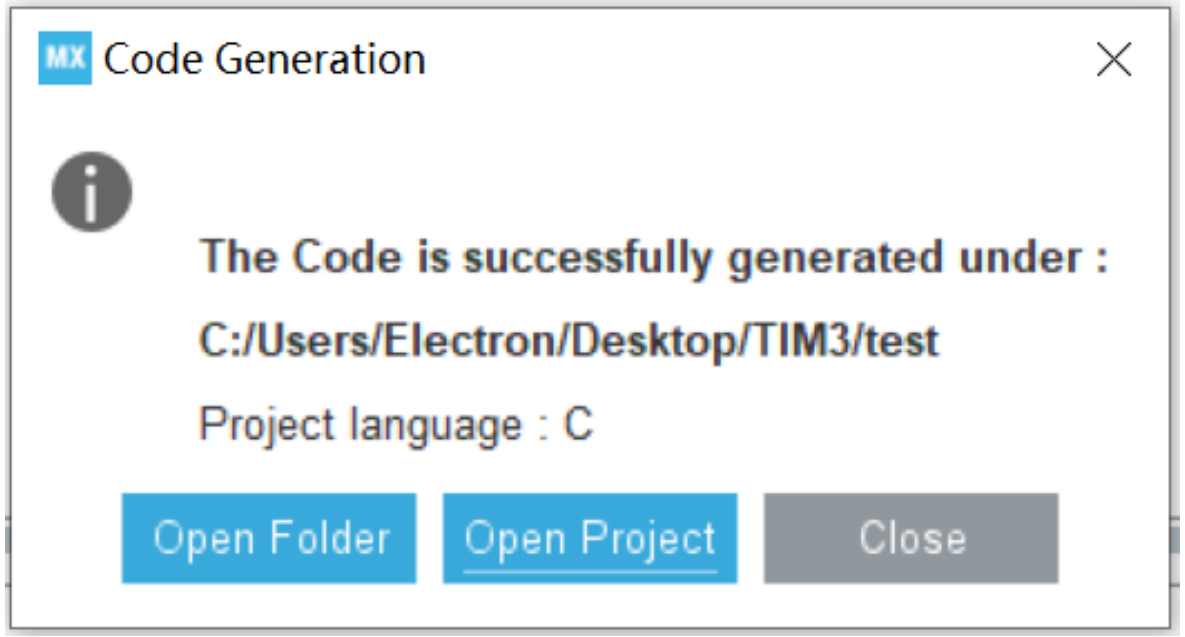
单击Project Manager选项页。

在工程选项中，设置Project Name、Project Location、Toolchain Floder Location，并选择Toolchain/IDE开发环境MDK-ARM。



3.1 STM32CubeMX

完成配置后，单击右上角GENERATE CODE，生成包含相关文件的MDK-ARM工程，生成工程成功后，单击Open Project，可以用Keil MDK软件打开进行编辑、调试。



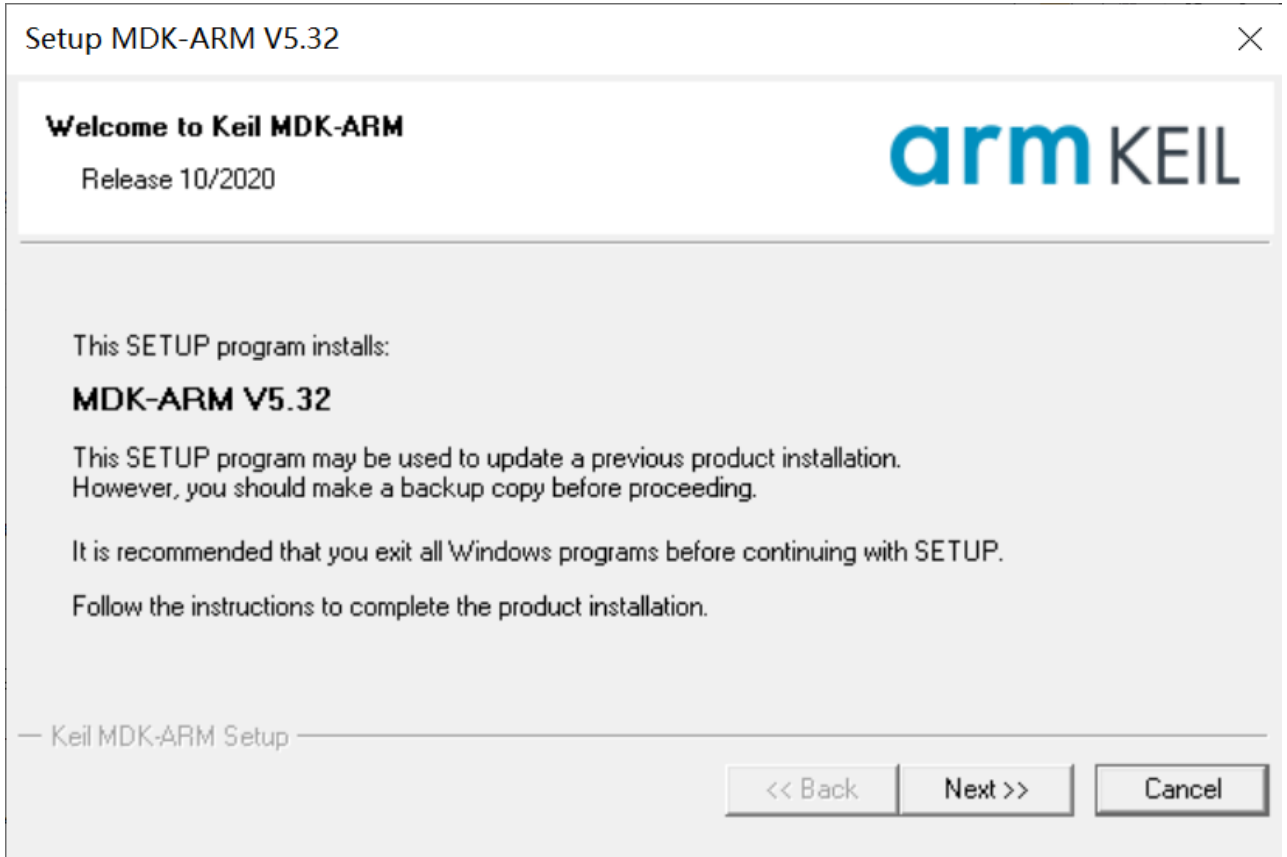
3.2 Keil MDK

Keil公司是一家业界领先的微控制器软件开发工具的独立供应商，制造和销售种类广泛的开发工具，包括ANSI C编译器、宏汇编程序、调试器、连接器、库管理器、固件和实时操作系统核心。

Keil MDK，也称为MDK-ARM、Realview MDK、I-MDK、uVision等，是Keil公司开发的ARM开发工具微控制器开发包（Microcontroller Development Kit，MDK），是用来开发基于ARM内核系列微控制器的嵌入式应用程序，适合不同层次的开发者使用，包括专业的应用程序开发工程师和嵌入式软件开发的入门者。

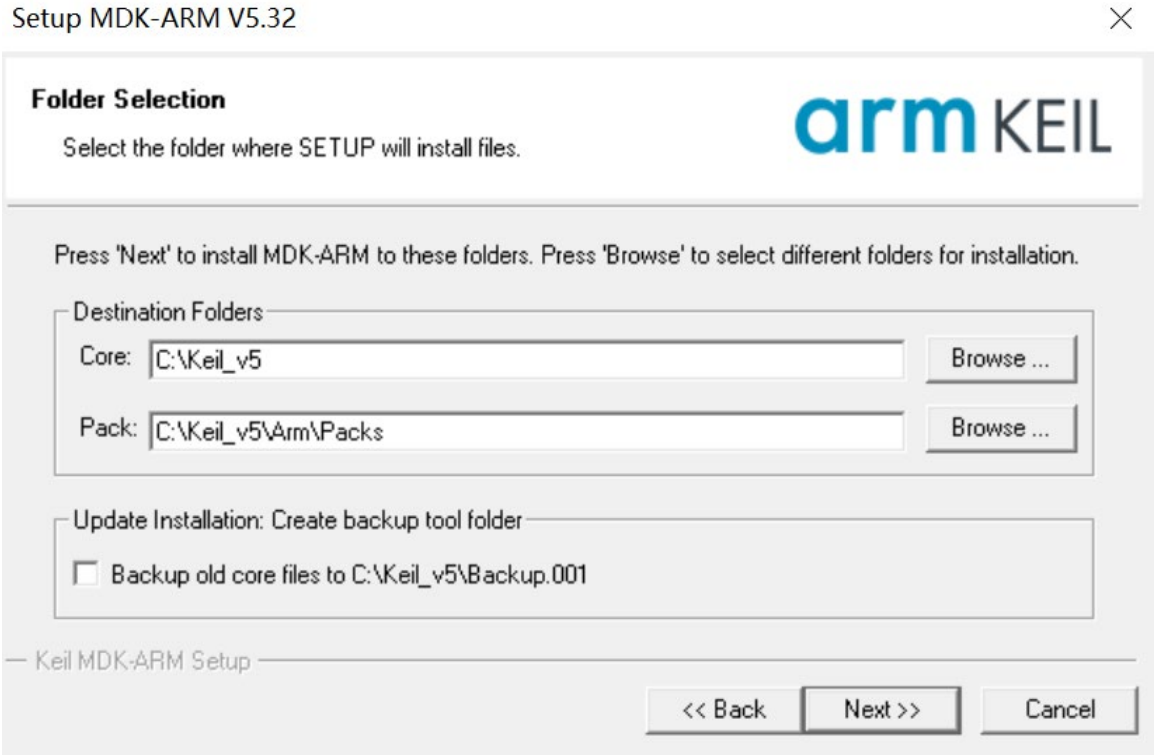
3.2 Keil MDK

运行Keil MDK安装软件，进入软件安装界面。



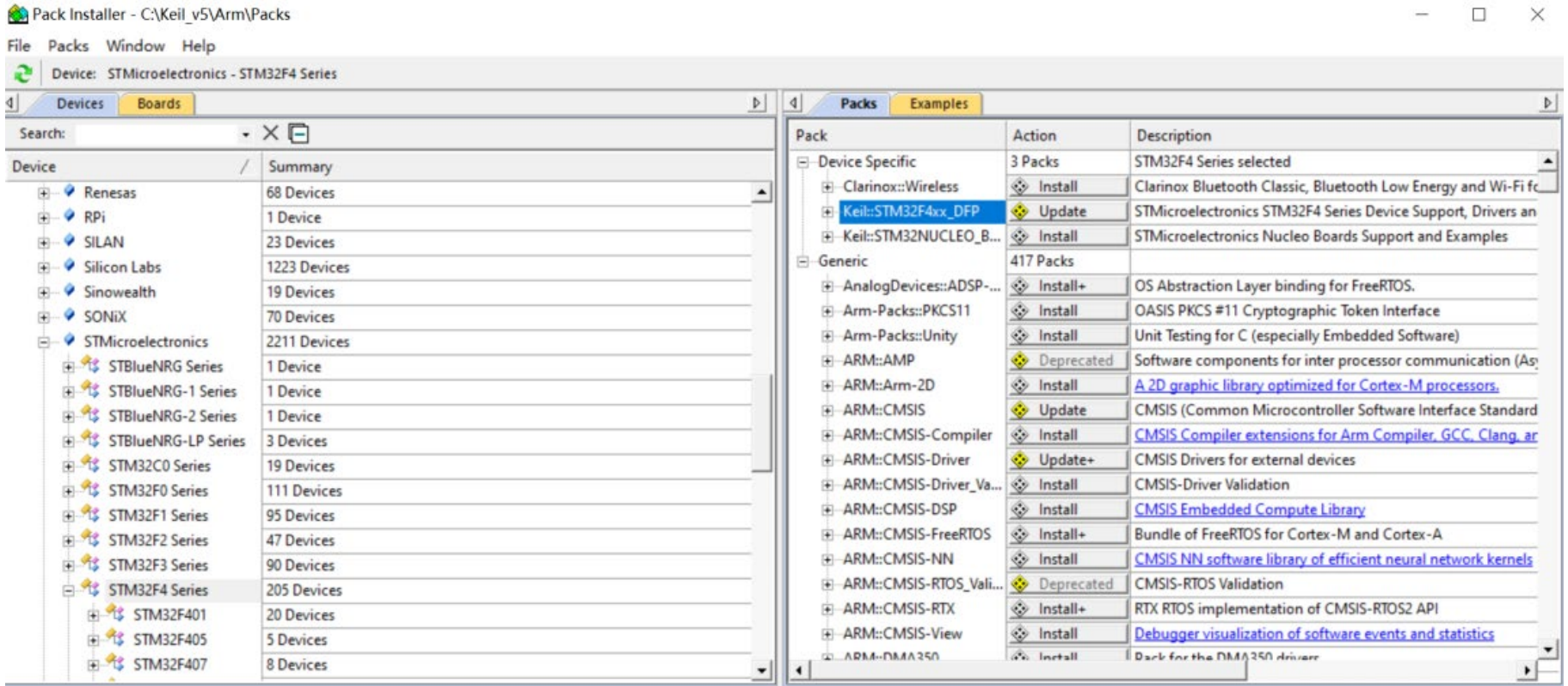
3.2 Keil MDK

设置合适的 Keil 安装路径，其中Core栏设置软件核心文件的安装路径，Pack栏设置设备支持文件、软件库、中间件和其他软件工具的安装路径，注意所有路径应在英文目录下。



3.2 Keil MDK

在软件工具栏中单击Pack Installer，或者单击软件菜单栏Project->Manage->Pack Installer，安装STM32F4系列Pack支持包，也可以单独下载STM32F4系列Pack支持包进行离线安装。



3.3 Proteus

Proteus是一款由英国Lab Center Electronics公司出版的电子设计自动化（Electronic design automation，EDA）工具软件，主要用于模拟硬件电路和编写嵌入式系统程序。Proteus不仅具有其他EDA工具软件的仿真功能，还能仿真单片机及外围器件，如51系列、AVR、PIC、ARM等常用主流单片机。它支持从原理图布图、代码调试到单片机与外围电路协同仿真，并可以一键切换到PCB设计，实现从概念到产品的完整设计。

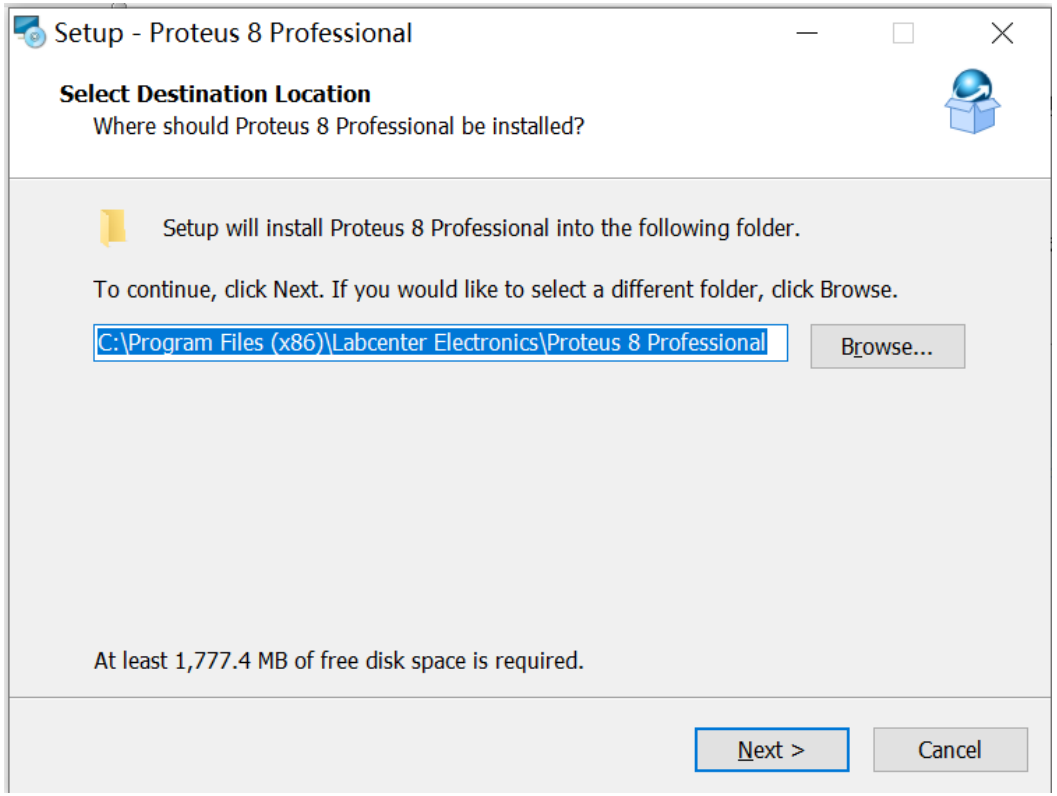
1. 概述

Proteus虚拟系统建模（Virtual System Modeling，VSM）为嵌入式系统的开发和调试提供了一种新的方法。具有原理图捕获和PCB布局的微控制器和混合模式SPICE仿真。

Proteus VSM是第一款为嵌入式设计弥合原理图和PCB之间差距的产品，在原理图内部提供基于微控制器的设计的系统级模拟。Proteus VSM具有数量较多的微控制器类型和外围设备，并且集成了测量和调试工具。

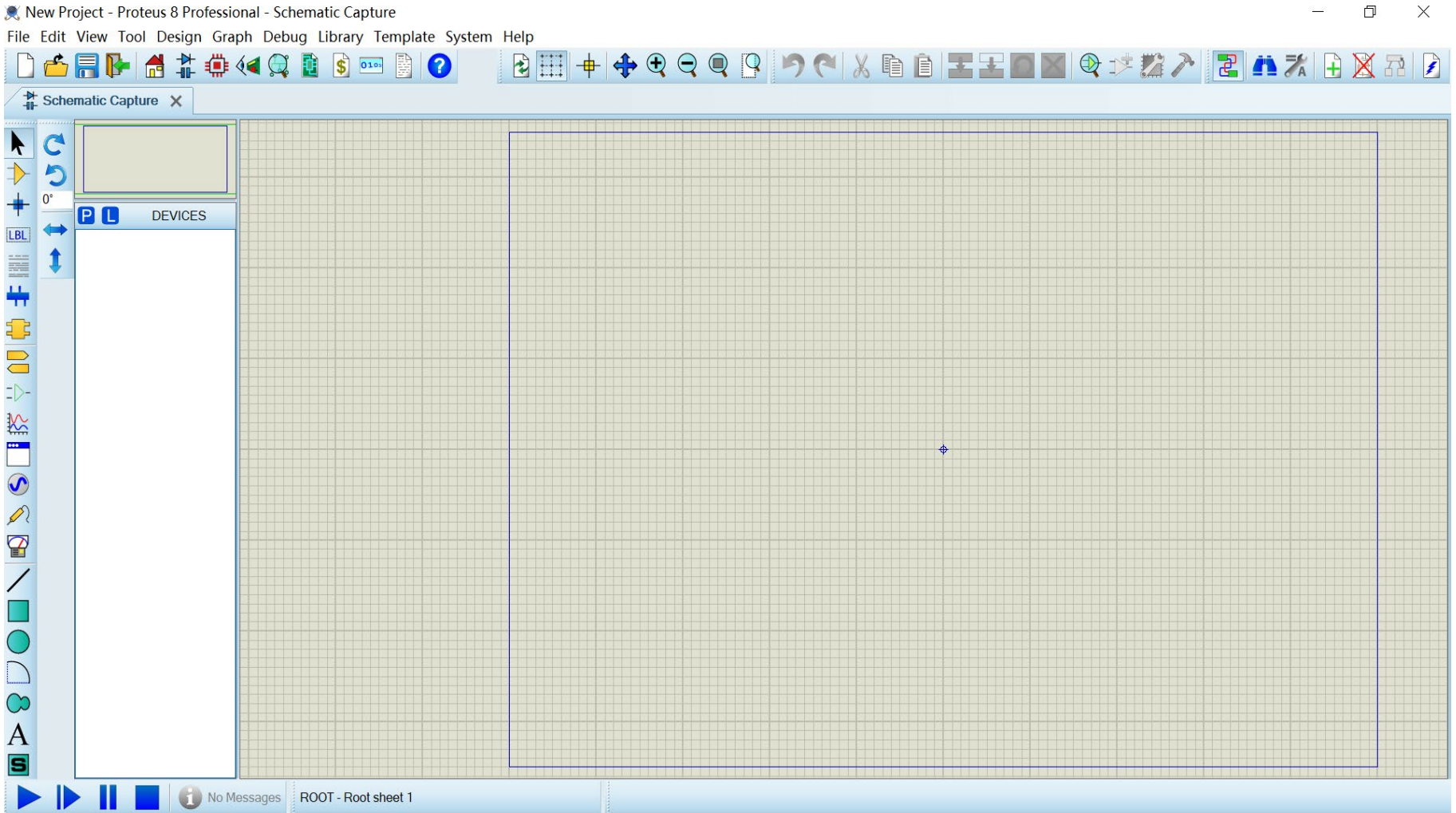
2. 程序安装

双击安装程序进入Proteus安装向导界面。单击Browse可以更改程序安装路径（安装路径须为英文路径）。

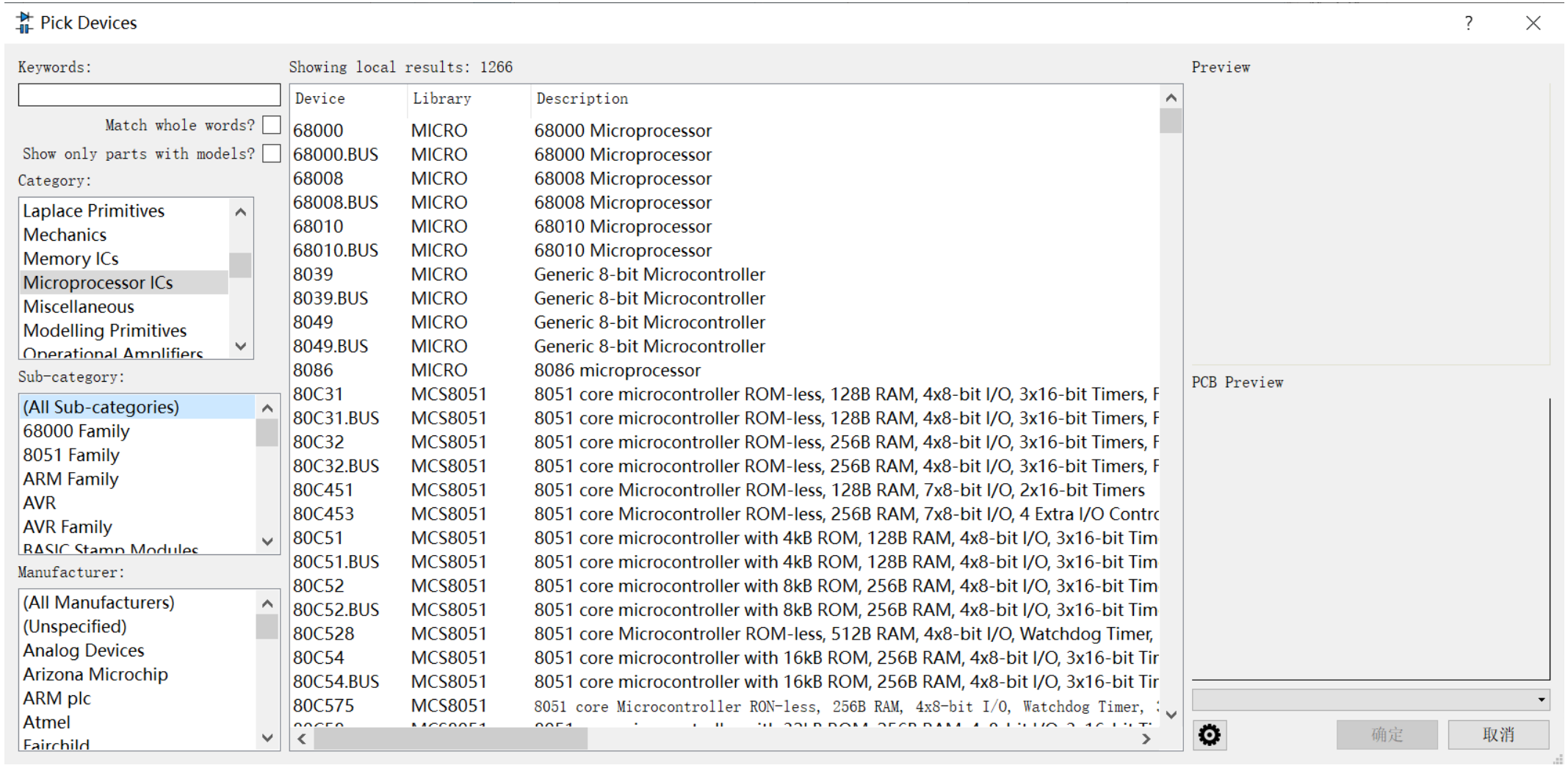


3. 建立STM32仿真模型

软件安装完成后，运行Proteus软件，选择新建原理图。

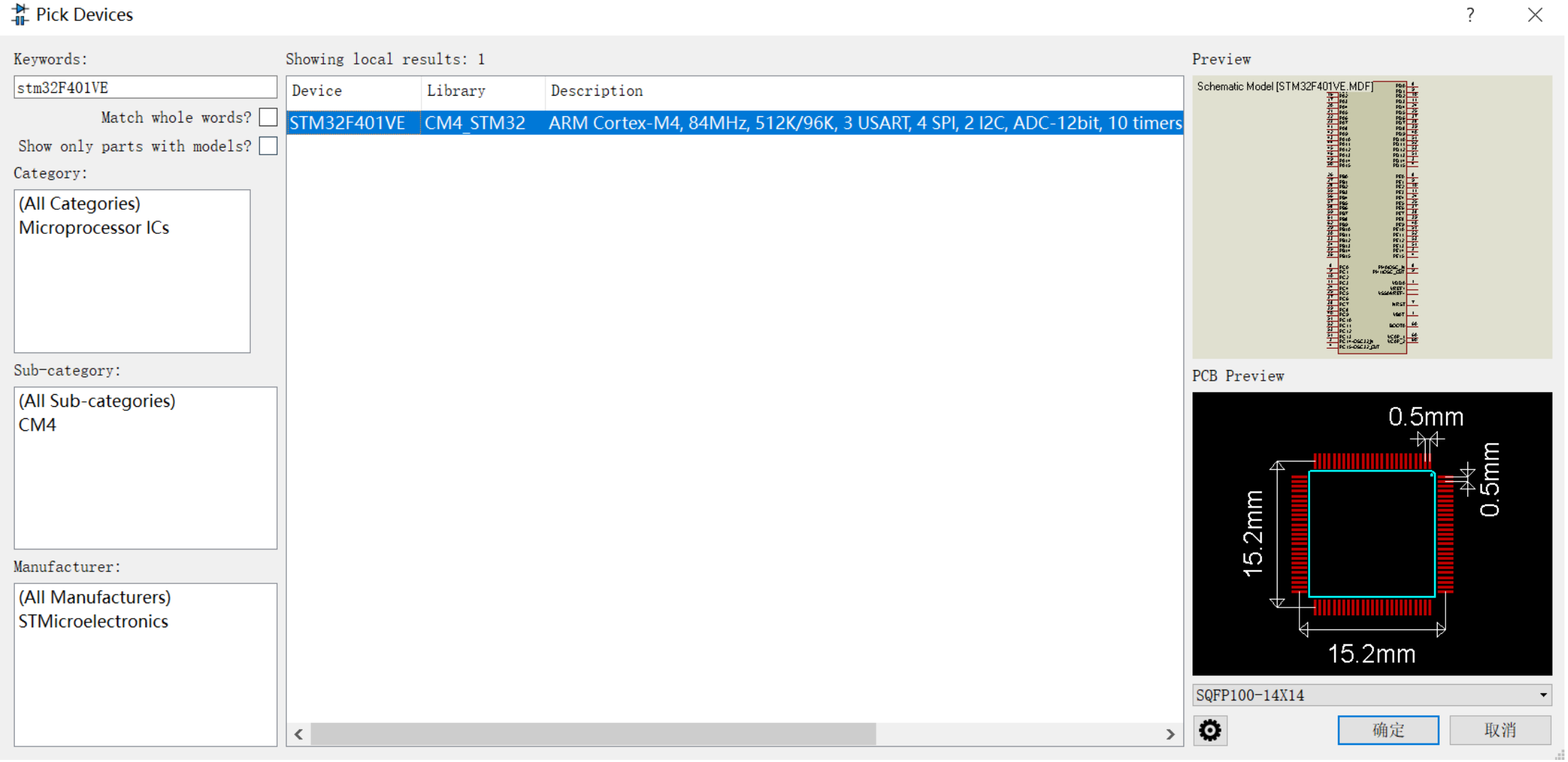


菜单栏选择Library->Pick Parts。

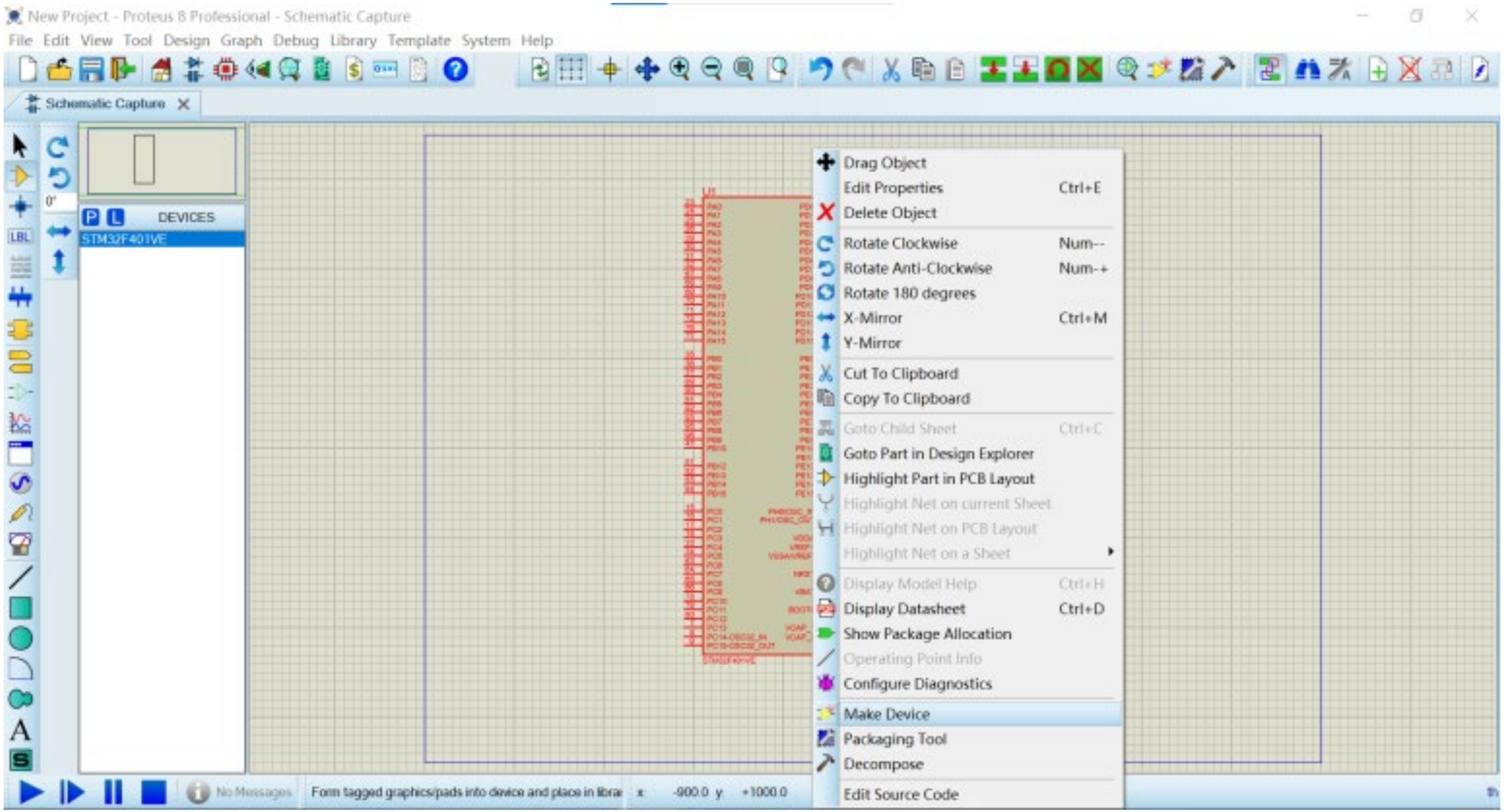


在Proteus中Cortex-M4芯片仿真模型目前没有STM32F407系列，仅有STM32F401系列，而由于STM32F407在性能、资源、功耗等方面更有优势且在应用市场使用广泛，在Proteus中采用STM32F401VE模型来建立STM32F407VET6的仿真模型。

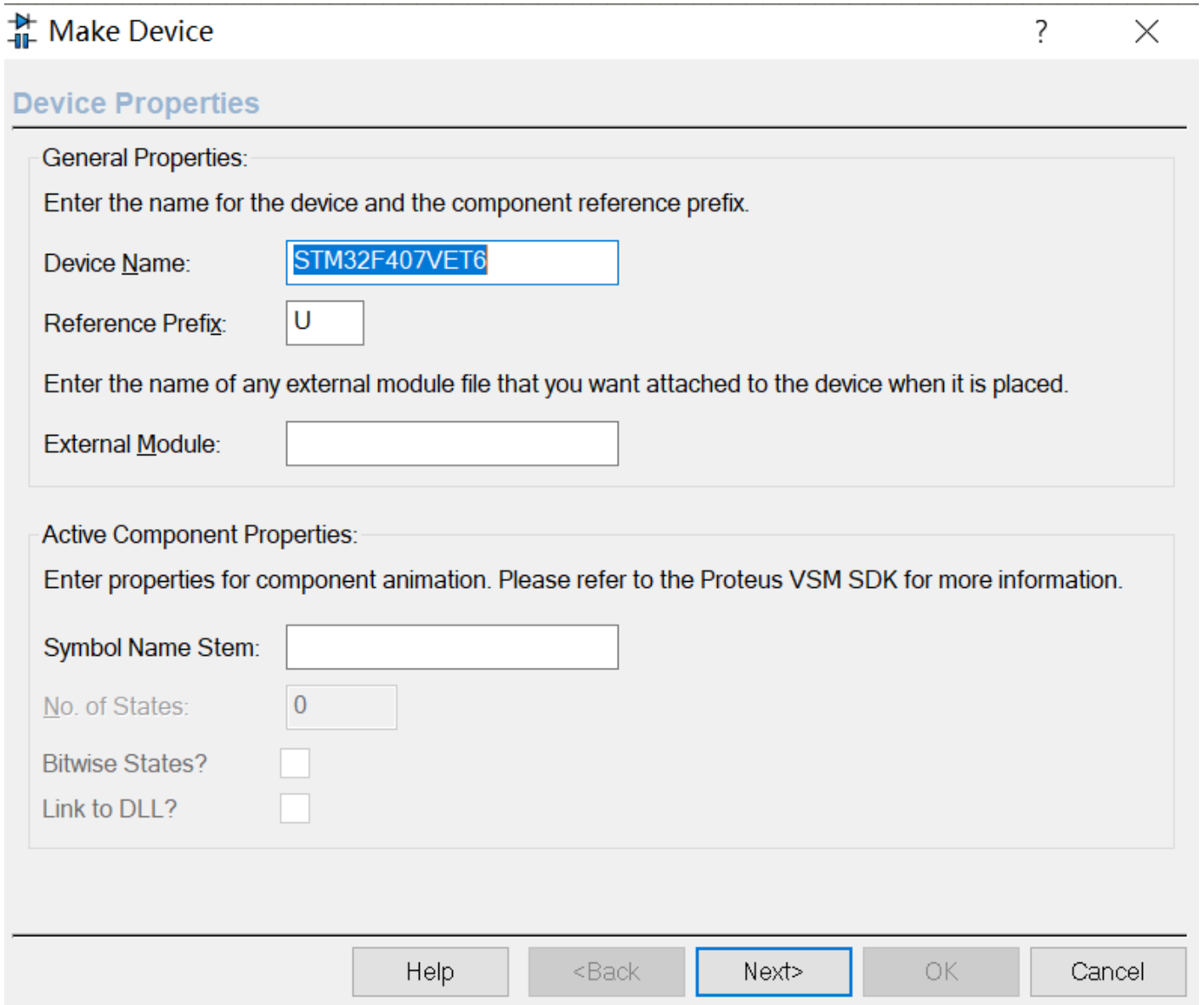
（1）在Keywords栏输入STM32F401VE，选择仿真模型。



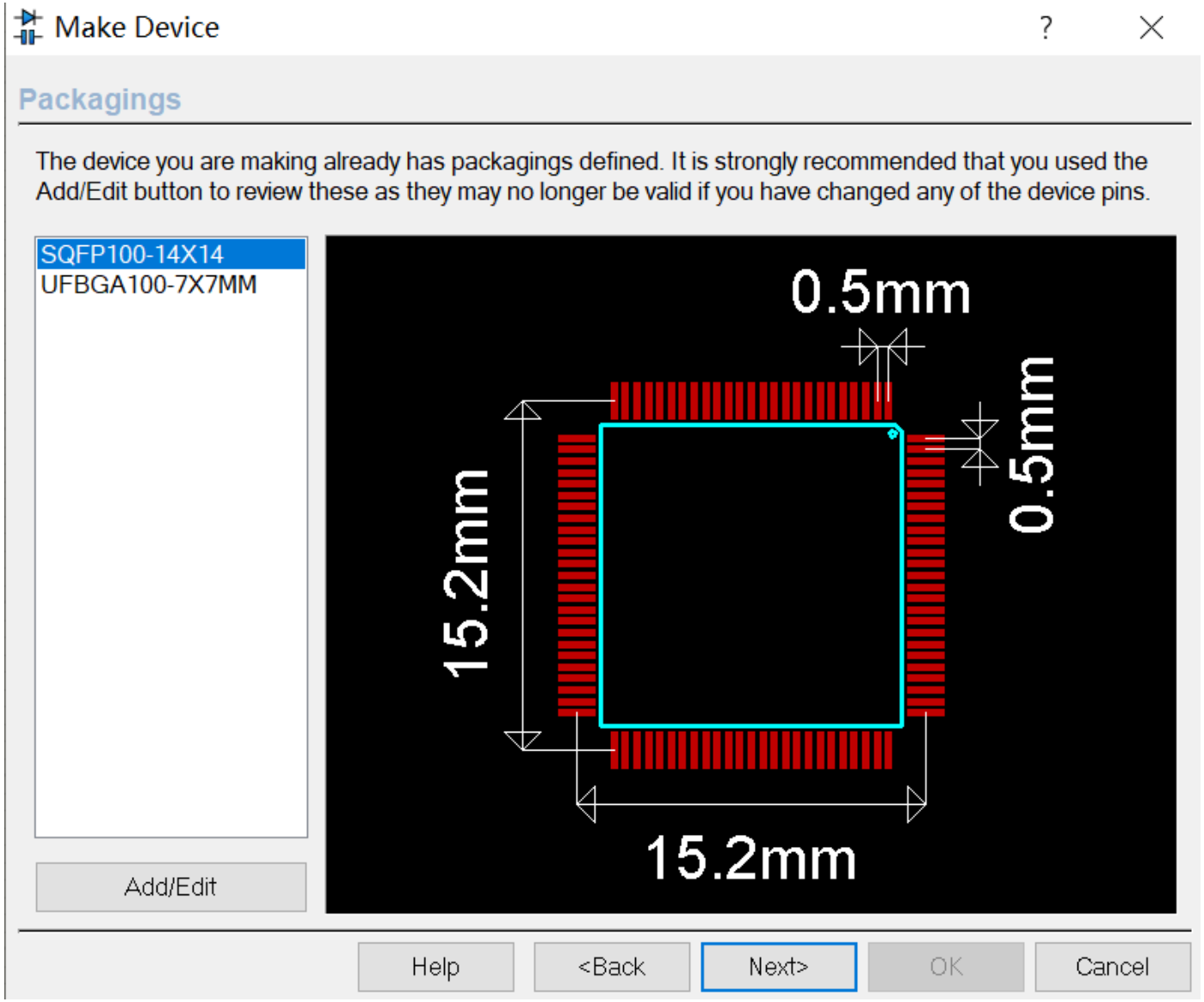
（2）将选择的仿真模型放入原理图，然后右键选择Make Device。



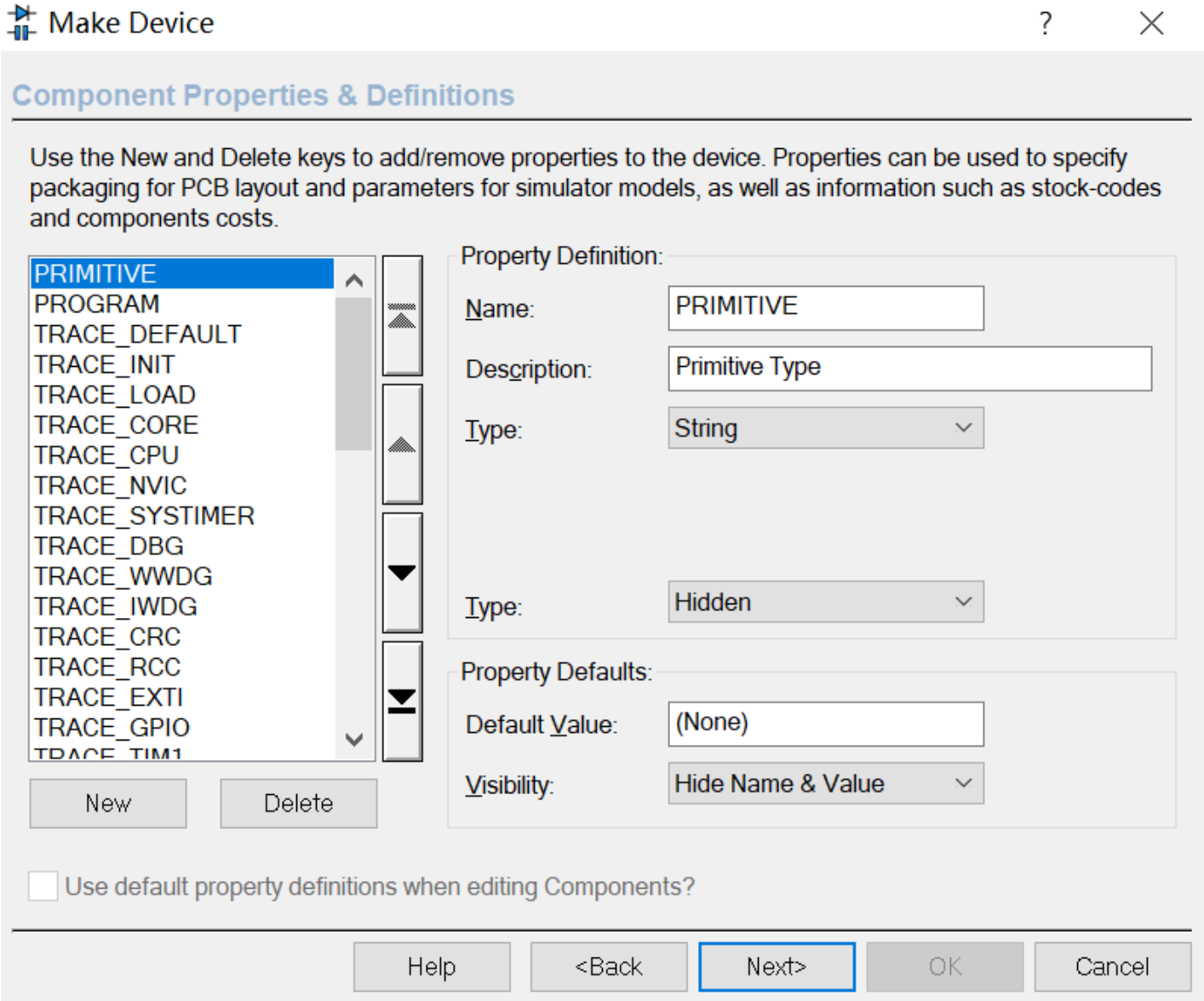
（3）器件属性配置，在Device Name栏输入：STM32F407VET6。



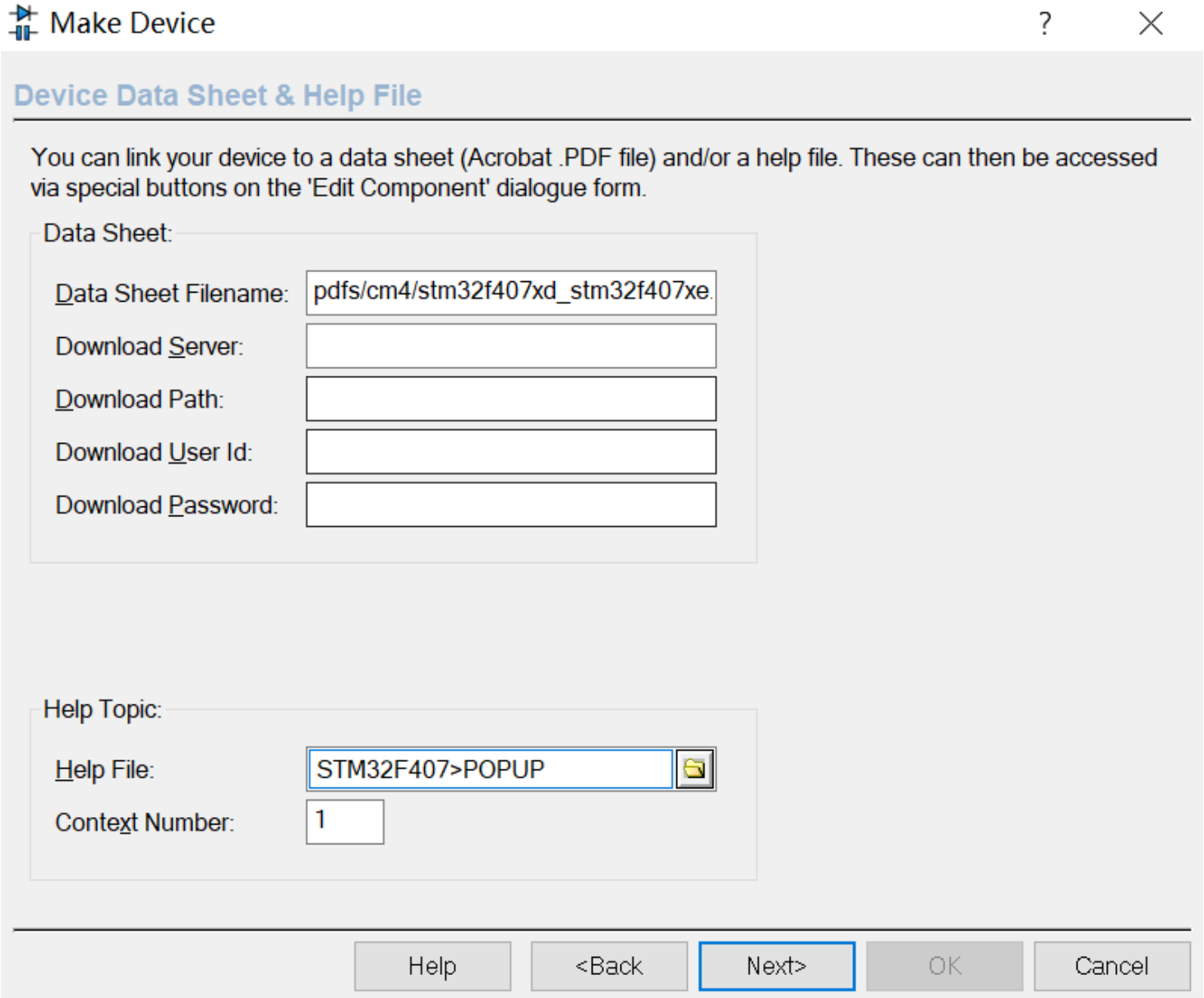
（4）选择器件封装，选择默认选型。



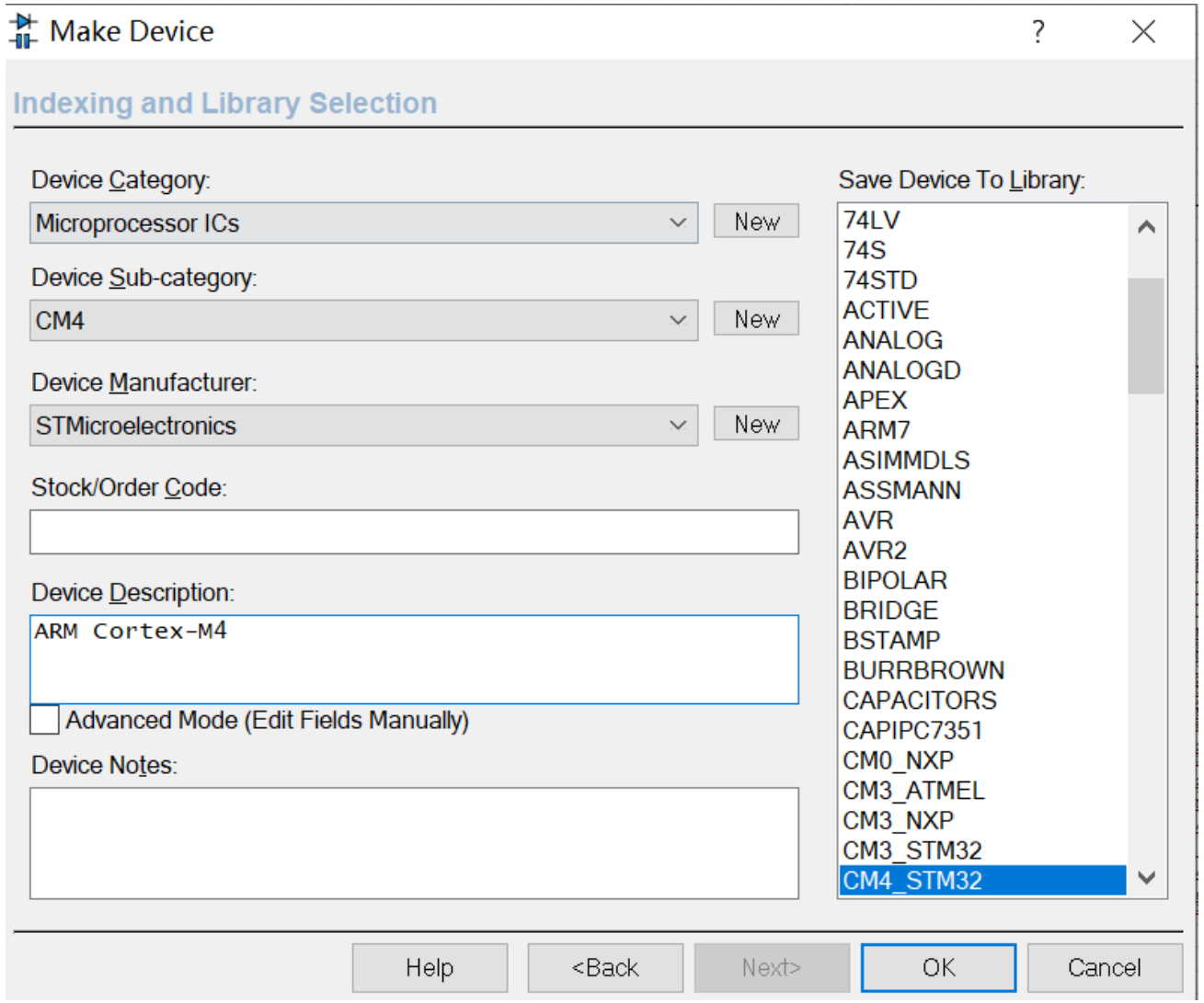
（5）选择器件属性和定义，采用默认值。



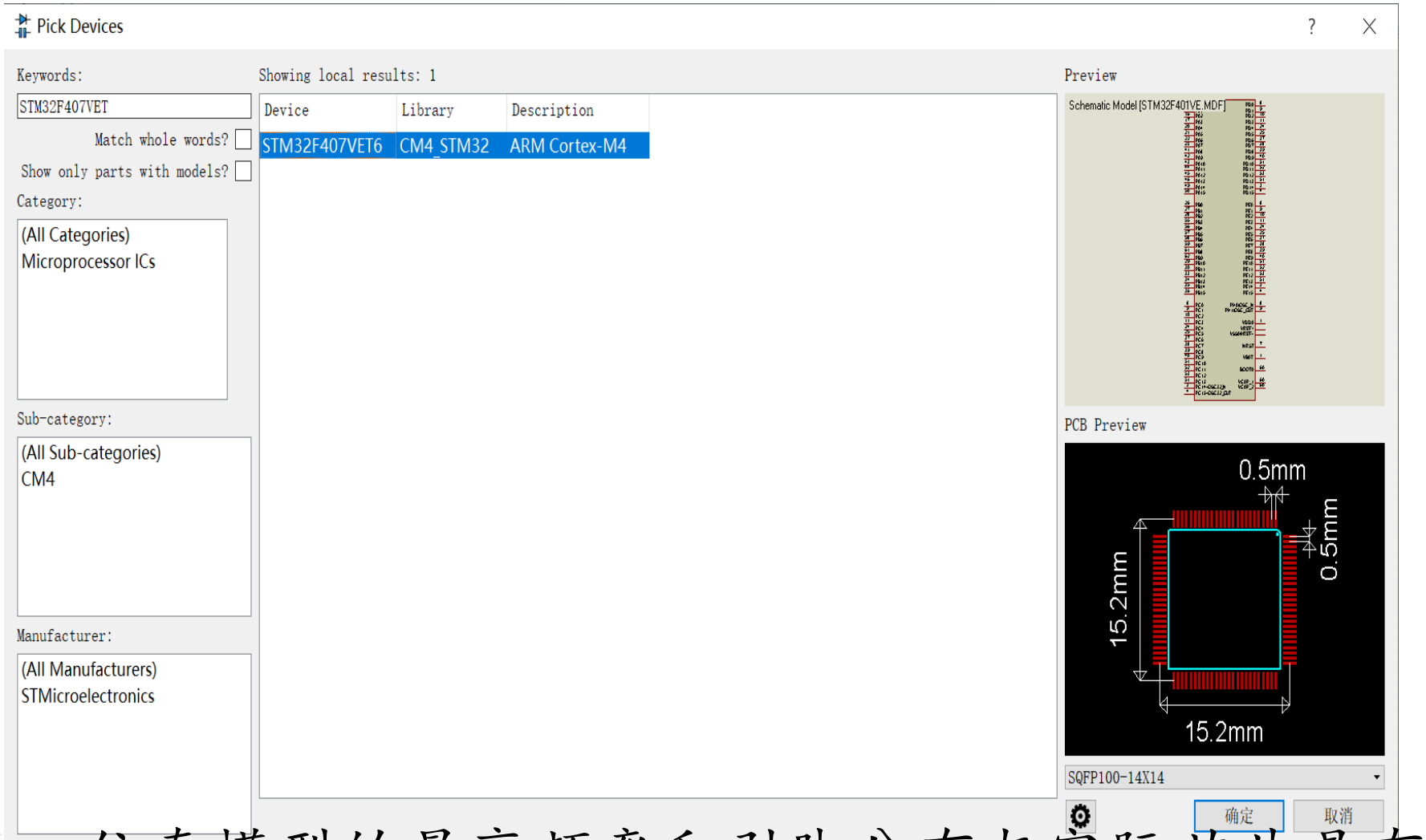
（6）器件参数表和帮助文件，可修改相应文件名称。



（7）索引和库选择，器件描述可以进行修改，在将器件保存到库中的选框里选择：CM4_STM32。



从库中选择元器件时，可以看到STMF407VET6已经在元件库中了。



注意：仿真模型的最高频率和引脚分布与实际芯片是有区别的，但基本功能都可以实现仿真，各章节的仿真实例均已进行了验证。

小结

首先介绍了ARM的相关概念、相关芯片的发展历史及其所适用的应用领域。然后，引出了STM32微控制器的概念及其核心板。最后，介绍了开发嵌入式应用系统所需要的工具，包括STM32CubeMX、Keil MDK和Proteus。

掌握ARM、STM32微控制器的有关概念，理解并掌握嵌入式系统常用开发工具，为后面学习STM32片内外设打下基础。